

Project Name: Machine Learning-Based Fraud Detection System for the Healthcare Sector

Author - Om Yadav

Mentor - Kunwar Saurabh Bisen

Date - June 2026

A Project Report

Field	Details
Student Name	Om Yadav
Student ID	O24MSD110085
Program	M.Sc. (Data Science) – V2
Project Title	MediGuard.AI – Machine Learning-Based Fraud Detection System for the Healthcare Sector
Guide / Mentor Name	Kunwar Saurabh Bisen
Institution	Chandigarh University, Mohali
Date of Submission	June 2026

Declaration

I hereby declare that the project report titled "**MediGuard.AI – Machine Learning-Based Fraud Detection System for the Healthcare Sector**" submitted to Chandigarh University is my original work. All sources of data and references used have been duly acknowledged. This work has not been submitted elsewhere for any other degree or diploma.

Signature: Om Yadav

Name: Om Yadav

Date: 26/06/2026

Acknowledgment

I would like to express my sincere gratitude to my mentor, **Kunwar Saurabh Bisen**, for their invaluable guidance, continuous support, and constructive feedback throughout this project. I also thank the faculty of the Department of Data Science, Chandigarh University, for providing the necessary resources and encouragement.

Special thanks to the **Qollabb platform** for enabling industry-academia collaboration and providing a structured framework for project execution. I am grateful to the open-source community for developing and maintaining the libraries and tools used in this project, including scikit-learn, XGBoost, LightGBM, LIME, Pandas, and Matplotlib.

I also acknowledge the dataset contributors on Kaggle for making the healthcare provider fraud detection dataset publicly available, which served as the foundation for this research.

Abstract

Healthcare provider fraud, waste, and abuse impose staggering systemic financial losses on the United States healthcare infrastructure, totaling tens of billions of dollars annually. Beyond financial damage, these illicit activities compromise patient clinical safety and inflate general insurance premiums. Traditional audit infrastructures heavily depend on retrospective rule-based scripts and slow manual reviews by domain experts. These methods fail to keep pace with the massive scale, multi-dimensional complexity, and evolving nature of modern insurance claims.

This project introduces **MediGuard.AI**, an automated machine learning system designed to detect fraudulent transactions submitted by healthcare providers. The problem is engineered as an imbalanced binary classification task. The system maps raw relational files—encompassing patient demographics, inpatient claims, outpatient claims, and provider matrices—into an integrated analytic environment.

Through feature engineering, a comprehensive pool of 135 features was built. These include per-provider financial aggregates, categorical code frequencies, relative diagnostic behaviors, and temporal duration matrices. To counter the stark class imbalance of the dataset (~9.35% fraud rate at the claim layer), multiple data sampling strategies were evaluated: Balanced Class Weights, Random Undersampling, and Synthetic Minority Oversampling Technique (SMOTE), combined with One-Hot and Target Response Encoding.

Four primary architectural paradigms were evaluated using cross-validated hyperparameter search grids: Decision Trees, Random Forests, XGBoost, and LightGBM. The **XGBoost Classifier combined with SMOTE Oversampling and Target Response Encoding** emerged as the top architecture. It achieved an **Area Under the Receiver Operating Characteristic (AUC-ROC) score of 0.9230**, an **F1-Score of 0.8560**, a **Log-Loss of 0.2870**, and an operational **False Negative Rate (FNR) of 0.1500**, restricting the overall misclassification footprint to just **3.52%**.

To satisfy clinical, administrative, and legal accountability, the system incorporates global interpretability using tree-based feature gain matrices alongside local interpretability via Local Interpretable Model-agnostic Explanations (LIME). This architecture creates a transparent fraud risk triage pipeline ready for seamless operational deployment.

Business Problem

Fraud, Waste, and Abuse have become one of the biggest concerns in the healthcare system of the United States and can be caused by different healthcare entities (3 Ps) viz, Patients (Insurance Subscriber/Beneficiary), Healthcare Providers (Hospitals, Physicians, Laboratories, Pharmacies, etc) and Payers (Insurance Providers like Medicare, Medicaid, and other agencies). Frauds can be caused by either a single entity or by a conspiracy of

multiple entities. In this case study, we primarily focus on the frauds caused by the Providers as part of insurance

claims submitted to the Insurance Agencies.

Many healthcare systems rely on domain experts to manually review insurance claims and detect potential frauds. There are hundreds of thousands to millions of claims data getting generated over time. Manual review of

the claims is very time-consuming and challenging. To reduce such effort, various machine learning techniques can be leveraged to automate the manual processing of the claims and identify the potential frauds. Such a system would certainly help in reducing the cost, manual effort, processing & review time, and other challenges in the system. This case study aims to solve the problem of detecting potential frauds in a healthcare system and

assist in making the healthcare system better.

Machine Learning Formulation of the Business Problem

Identify the potential frauds made by the Providers, in the insurance claims submitted to the Insurance Agencies.

The problem can be projected into a binary classification task of determining whether the claim record(s) submitted by the Providers is/are fraudulent or legitimate. Given the data related to the Providers and the claim submitted by them, a Machine Learning Model can be built to determine whether a claim record is fraudulent or not.

Business Constraints

Table of Contents

Chapter No.	Title	Page
1	Introduction	1
2	System Analysis	15
3	System Design	25
4	System Implementation	40
5	Testing	60
6	Result and Discussion	70
7	Conclusion and Future Enhancement	80
8	Bibliography and References	85
A	Appendix A – Dataset Description	88
B	Appendix B – Code Implementation Snippets	92
C	Appendix C – Additional Figures and Tables	100

List of Figures

Figure No.	Figure Title	Page
1.1	Healthcare Fraud Detection Workflow	5
2.1	Existing vs. Proposed System Flow	18
3.1	Proposed System Architecture	26
3.2	Data Flow Diagram (Level 0)	30
3.3	Data Flow Diagram (Level 1)	31
3.4	Entity-Relationship (ER) Diagram	33
4.1	Data Merge Pipeline	42

Figure No.	Figure Title	Page
4.2	Distribution of 'ClnAdmitDiagnosisCode' – Top 50	45
4.3	Distribution of 'ClnAdmitDiagnosisCode' – Bottom 50	45
4.4	Distribution of 'DiagnosisGroupCode' – Top 50	46
4.5	Distribution of 'DiagnosisGroupCode' – Bottom 50	46
4.6	Histogram/PDF of 'ClaimSettlementDelay' Feature	47
4.7	Box Plot of 'ClaimSettlementDelay' Feature	47
4.8	Histogram/PDF of 'TreatmentDuration' Feature	48
4.9	Box Plot of 'TreatmentDuration' Feature	48
4.10	Histogram/PDF of 'InscClaimAmtReimbursed' Feature	49
4.11	Box Plot of 'InscClaimAmtReimbursed' Feature	49
4.12	Feature Correlation Heatmap	52
4.13	Top 10 Procedures Involved in Healthcare Fraud	55
4.14	Top 10 Diagnosis Involved in Healthcare Fraud	56
4.15	State-wise Beneficiary Distribution	57
4.16	Provider Fraud Class Distribution	58
5.1	Confusion Matrix Structure	62
5.2	Model Comparison – AUC and F1-Score	68
6.1	Feature Importance – Top 15 Features (XGBoost)	73
6.2	LIME Interpretation for a Query Point	75
6.3	ROC-AUC Curves for All Models	76
6.4	Precision-Recall Curves for All Models	77
6.5	Confusion Matrix – XGBoost with SMOTE	78

List of Tables

Table No.	Table Title	Page
1.1	Project Timeline (Gantt Chart Summary)	12
2.1	Functional vs. Non-Functional Requirements	20
2.2	Feasibility Analysis Matrix	22
3.1	Dataset File Description	35

Table No.	Table Title	Page
3.2	Features List – Beneficiary Data	36
3.3	Features List – Inpatient/Outpatient Data	37
3.4	Features List – Mapping Data	38
3.5	Performance Metrics Summary	39
4.1	Environment and Tools Used	41
4.2	Percentage of Null Values (Top Features)	44
4.3	Feature Engineering – New Features Created	53
5.1	Hyperparameter Tuning Grid – XGBoost	64
5.2	Hyperparameter Tuning Grid – Random Forest	65
5.3	Hyperparameter Tuning Grid – LightGBM	66
5.4	Test Results – Confusion Matrix Values	67
6.1	Model Performance Comparison Table	72
7.1	Objectives vs. Achievement Mapping	82
C.1	Complete Hyperparameter Tuning Results	101
C.2	GitHub Repository Structure	103

Chapter 1: Introduction

1.1 Project Background & Domain Magnitude

Fraud, Waste, and Abuse (FWA) within the United States healthcare financial ecosystem represent systemic threats that drain substantial financial assets out of public programs and private clinical networks. The scale of this problem is vast. The National Health Care Anti-Fraud Association (NHCAA) indicates that fraud routinely consumes between 3% and 10% of total national healthcare expenditures. Given that total US health spending runs into trillions of dollars, these percentages translate directly to tens of billions of dollars lost each single fiscal year.

These massive losses have compounding negative downstream real-world effects: they drive up general insurance premiums for consumers, exhaust public capital reserves within Medicare and Medicaid networks, and direct vital financial resources away from modern patient care.

The structural taxonomy of healthcare insurance transactions centers on interactions among three distinct functional entities, conceptually referred to as the **3 Ps**:

1. **Patients:** The primary insured beneficiaries who receive clinical medical care. Fraud profiles at this layer typically manifest as third-party identity theft, falsified application profiles, or systematic illicit prescription aggregation.
2. **Providers:** The operational healthcare delivery infrastructure, which includes large-scale clinical networks, regional community hospitals, specialized diagnostic labs, physical health practitioners, and pharmacies.
3. **Payers:** The fiscal underwriting and settlement systems, which encompass public government health administrations (such as Medicare and Medicaid) alongside corporate private insurance entities.



While fraudulent activity can manifest across any individual vector or through complex, multi-party collusive structures, **MediGuard.AI focuses explicitly on fraud initiated by Providers.**

Provider-driven fraud is exceptionally costly due to its systemic nature. Rather than isolating single claims, it targets persistent billing behaviors across large volumes of transactional insurance claims submitted to payers.

Common systemic mechanisms of provider fraud include:

- **Upcoding:** Intentionally substituting standard medical billing codes with higher-intensity procedural codes to secure inflated reimbursement rates for simple clinical visits.
- **Phantom Billing:** Formally filing transaction logs for medical procedures, diagnostic assessments, or prescriptions that were never actually performed or distributed.
- **Unbundling:** Separating components of a single integrated surgical or diagnostic event into dozens of independent line-item claims to circumvent unified package discount constraints.
- **Medically Unnecessary Services:** Ordering high-cost specialized diagnostic scans (e.g., unnecessary MRIs) solely to maximize clinical facility asset throughput and subsequent financial reimbursement.

1.1.1 The Magnitude of Healthcare Fraud

Healthcare fraud is a massive problem in the United States. According to the National Health Care Anti-Fraud Association (NHCAA), healthcare fraud accounts for approximately 3% to 10% of total healthcare expenditures in the U.S., which translates to tens of billions of dollars in losses annually. The Government Accountability Office (GAO) has designated Medicare and Medicaid as "high-risk" programs due to their vulnerability to fraud, waste, and abuse.

The impact of healthcare fraud extends beyond mere financial losses:

- **Increased Insurance Premiums:** Insurance agencies must recover their losses by raising premium rates, making healthcare less affordable for everyone.
- **Compromised Quality of Care:** Fraudulent activities divert resources away from legitimate patient care, potentially compromising the quality of healthcare services.
- **Patient Safety Risks:** Unnecessary procedures, counterfeit medications, and substandard medical services can directly harm patients.
- **Erosion of Trust:** Widespread fraud undermines public trust in the healthcare system and government programs.

1.1.2 Current Challenges in Fraud Detection

Many healthcare systems rely on domain experts to manually review insurance claims and detect potential frauds. However, this approach faces several fundamental challenges:

1. **Volume:** There are hundreds of thousands to millions of claims generated over time. The sheer volume makes thorough manual review impractical.
2. **Complexity:** Fraudulent claims are often designed to appear legitimate, requiring deep domain expertise to identify suspicious patterns.
3. **Time Sensitivity:** Delays in fraud detection mean that fraudulent payments have already been processed, making recovery difficult.
4. **Inconsistency:** Different reviewers may apply different criteria, leading to inconsistent fraud detection outcomes.
5. **Cost:** Employing a large team of medical domain experts for manual review is prohibitively expensive.

To address these challenges, various machine learning techniques can be leveraged to automate the processing of claims and identify potential frauds more efficiently and consistently. Such an automated system would significantly reduce costs, manual effort, processing and review time, and other operational challenges in the healthcare system.

Figure 1.1: Healthcare Fraud Detection Workflow – The conceptual workflow illustrates the journey of insurance claims from submission to fraud detection, encompassing data ingestion, preprocessing, model training, evaluation, and deployment stages.

1.2 Problem Statement

The healthcare industry generates an enormous volume of insurance claims data daily. Payers (insurance companies and government agencies) are responsible for reviewing these claims to identify fraudulent activities before processing payments. Currently, this review process is predominantly manual, relying on the expertise of medical professionals and claims adjusters.

However, given the scale of claims data — often running into millions of records per month — manual review is:

- **Time-Consuming:** A single claim can take hours to thoroughly investigate.
- **Resource-Intensive:** Requires a large workforce of specialized professionals.
- **Prone to Error:** Human reviewers may miss subtle patterns indicative of fraud.
- **Inconsistent:** Different reviewers may reach different conclusions on similar cases.

There is a pressing need for an **automated, intelligent system** that can:

- Flag potentially fraudulent claims with high accuracy.
- Maintain a low false negative rate (minimizing missed frauds).
- Provide interpretable reasoning for each prediction (essential for legal proceedings).
- Scale efficiently with growing data volumes.
- Adapt to evolving fraud patterns over time.

This project aims to solve the problem of detecting potential frauds in the healthcare system using machine learning techniques, thereby assisting in making the healthcare system more efficient, cost-effective, and trustworthy.

1.3 Machine Learning Formulation of the Business Problem

The core problem can be formally stated as follows:

Identify potential frauds committed by Healthcare Providers in insurance claims submitted to Insurance Agencies.

This problem can be projected as a **binary classification task**:

- **Positive Class (1):** The claim record submitted by the Provider is **fraudulent**.
- **Negative Class (0):** The claim record submitted by the Provider is **legitimate**.

Given the data related to Providers and the claims submitted by them, a Machine Learning Model can be built to determine whether a claim record is fraudulent or not. Mathematically, we seek to learn a function $f: \mathbb{R}^n \rightarrow \{0,1\}$ that maps a feature vector $\mathbf{x} \in \mathbb{R}^n$ (representing claim and provider attributes) to a binary label $y \in \{0,1\}$ indicating fraud status.

The model outputs not only the binary prediction but also a class probability $P(y=1|\mathbf{x})$, allowing stakeholders to assess the confidence level of each prediction and prioritize high-probability cases for further investigation.

1.4 Business Constraints

The fraud detection system operates under several critical constraints that shape the choice of algorithms, evaluation metrics, and deployment strategy:

1.4.1 No Strict Latency Requirement

Unlike real-time systems such as credit card fraud detection (which must respond in milliseconds), healthcare claims processing does not have strict low-latency requirements. The current manual process takes days or weeks per claim, so a system that processes claims in minutes or hours represents a significant improvement. However, the processing time should still be reasonable, typically within a few hours for batch processing of large claim volumes.

1.4.2 Interpretability (Critical)

Interpretability is of paramount importance for this application. When a Provider is identified as fraudulent, there could be serious legal and financial consequences, including:

- Legal proceedings and potential criminal charges.
- Exclusion from government healthcare programs (e.g., Medicare/Medicaid).
- Reputational damage and loss of business.
- License revocation.

Therefore, the Machine Learning Model must provide proper justification for its predictions. If a legitimate Provider is incorrectly flagged as fraudulent and subjected to legal proceedings, they may:

- Restrict their facilities and cooperation with government agencies.
- Limit access to healthcare services in underserved areas.
- Pursue legal action against the insurance agency.

The interpretability requirement makes "black-box" models less suitable unless accompanied by robust explainability frameworks.

Business Constraints

1. **No Latency Requirement:** As the current process of manually labeling the claims as fraudulent or not, takes much time, there won't be a strict need to have a low-latency system for fraud detection. However, the time taken should be reasonable depending on the business need.
2. **Interpretability:** Interpretability is very important as the machine learning model needs to justify why it classifies a claim as fraudulent or not. There could be legal proceedings involved on the Providers after identifying a fraudulent claim filed by them. Hence, it is of utmost importance that the Machine Learning Model gives proper justification in case of detecting fraud.

If a legitimate Provider is detected as fraud and has to undergo legal proceedings, it may plan to restrict its facility and cooperation with the Government, and the reachability of the healthcare systems will be compromised.
3. **Errors:** False Negative Rate (FNR) should be as low as possible. In some healthcare systems which conduct some manual review after the machine classification, a slightly higher False Positive Rate (FPR) is acceptable. However, FNR should always be minimum, otherwise, the actual fraud Providers would escape and continue to impact the healthcare system.
4. **Class Probabilities:** It would be better to provide class probabilities so that the user can know how probable a particular fraudulent claim is.

1.4.3 Error Prioritization — Minimizing False Negatives

In classification tasks, two types of errors can occur:

1. **False Positive (FP):** A legitimate claim is incorrectly flagged as fraudulent.

2. **False Negative (FN):** A fraudulent claim is incorrectly classified as legitimate.

In many healthcare systems that conduct manual review after machine classification, a slightly higher False Positive Rate (FPR) is acceptable — the flagged claims can be reviewed by human experts who can overturn incorrect fraud predictions. However, the **False Negative Rate (FNR) must always be minimized**. If fraudulent claims escape detection, the fraud continues unchecked, causing ongoing financial losses and compromising the quality of healthcare.

The cost of a false negative is significantly higher than the cost of a false positive in this context. Therefore, model selection and tuning must prioritize minimizing FNR, even at the cost of a moderate increase in FPR.

1.4.4 Class Probabilities

The system should provide class probabilities (confidence scores) alongside binary predictions. This allows:

- Prioritization of high-confidence fraudulent claims for immediate investigation.
- Triage of borderline cases for deeper analysis.
- Calibration of decision thresholds based on business needs and resource availability.
- Better transparency in the decision-making process.

1.5 Objectives

The primary objectives of this project are:

1. **Data Preprocessing and Analysis:** To preprocess and analyze healthcare claims data from multiple sources, identifying patterns, anomalies, and features indicative of fraudulent activity.
2. **Machine Learning Model Development:** To develop, train, and evaluate multiple machine learning models for binary classification of fraudulent vs. legitimate claims, including Decision Tree, Random Forest, XGBoost, and LightGBM classifiers.
3. **Performance Optimization with FNR Minimization:** To optimize model performance with a specific emphasis on minimizing the False Negative Rate (FNR) while maintaining strong overall accuracy metrics.
4. **Model Interpretability:** To provide comprehensive model interpretability using both global techniques (feature importance analysis) and local techniques (LIME — Local Interpretable Model-agnostic Explanations).
5. **Deployment Readiness:** To prepare the best-performing model for real-world deployment, including model serialization, documentation, and integration guidelines.
6. **Imbalanced Learning:** To effectively handle the severe class imbalance inherent in fraud detection datasets (where fraudulent claims typically constitute less than 10% of all claims) using appropriate sampling and weighting techniques.

1.6 Scope of the Project

In Scope

The project covers the following aspects:

- Fraud detection in **inpatient and outpatient insurance claims** submitted by healthcare Providers.

- Data sourced from Kaggle's **"Healthcare Provider Fraud Detection Analysis"** dataset, which includes beneficiary demographics, inpatient claims, outpatient claims, and provider mapping data.
- Binary classification of claims as fraudulent or legitimate.
- Implementation of four machine learning models: Decision Tree, Random Forest, XGBoost, and LightGBM.
- Two encoding schemes: One-Hot Encoding and Response Encoding.
- Multiple sampling strategies: raw data, balanced class weights, random undersampling, and SMOTE oversampling.
- Model interpretability using feature importance (global) and LIME (local).
- Performance evaluation using a comprehensive set of metrics including AUC, Log-Loss, F1-Score, FNR, FPR, Balanced Accuracy, and MCC.

Out of Scope

The following areas are explicitly outside the current project scope:

- **Real-time fraud detection:** The system is designed for batch processing, not real-time transaction screening.
- **Deep Learning solutions:** Neural network architectures such as MLPs, CNNs, RNNs, or Transformers are not explored.
- **Multi-language deployment:** The system currently supports English-language data only.
- **Multi-entity fraud detection:** The system focuses on Provider fraud only, not Patient or Payer fraud.
- **Unsupervised anomaly detection:** Novel fraud patterns not present in training data are not addressed.
- **Cross-domain applicability:** The system is tailored for healthcare claims data and would require significant adaptation for other domains.

1.7 Project Timeline

The project was executed over a structured 12-week timeline, divided into distinct phases as shown in Table 1.1.

Table 1.1: Project Timeline (Gantt Chart Summary)

Week	Activity	Deliverables
Week 1–2	Initial Planning and Requirement Analysis	Project proposal, requirement specification
Week 3–4	Data Collection and Understanding	Data acquisition, exploratory data analysis
Week 5–6	Data Preprocessing and Feature Engineering	Cleaned dataset, engineered features
Week 7–8	Model Development and Training	Trained models, hyperparameter tuning results

Week	Activity	Deliverables
Week 9–10	Solution Integration and Testing	Model evaluation, performance comparison
Week 11–12	Deployment and Monitoring	Final model, documentation, report

1.8 Report Organization

This report is organized as follows:

- **Chapter 2 (System Analysis):** Analyzes the existing system, identifies gaps, and defines requirements for the proposed solution.
- **Chapter 3 (System Design):** Presents the system architecture, data flow diagrams, ER diagrams, and detailed design specifications.
- **Chapter 4 (System Implementation):** Describes the development environment, data preprocessing, feature engineering, EDA, and model training implementation.
- **Chapter 5 (Testing):** Details the testing strategy, hyperparameter tuning process, and test results.
- **Chapter 6 (Result and Discussion):** Presents and discusses the performance of all models, including interpretability analysis.
- **Chapter 7 (Conclusion and Future Enhancement):** Summarizes findings and outlines directions for future work.
- **Chapter 8 (Bibliography and References):** Lists all sources and references used.
- **Appendices:** Provide supplementary material including dataset descriptions, code snippets, and additional figures.

Chapter 2: System Analysis

2.1 Existing System

Legacy healthcare fraud detection depends heavily on retrospective human review and rigid, static rule-based checks. In these systems, claims are processed through simple validation scripts that look for obvious compliance errors, such as duplicate transaction IDs or invalid patient demographic keys. Any claim that triggers a rule is sent to manual compliance teams for review.

This approach has major structural drawbacks. Human analysis scales poorly; as claim volumes grow, compliance teams become bottlenecks, forcing organizations to audit only small fractions of total transactions. Static rules are easily bypassed by sophisticated bad actors who alter their billing patterns to stay just under automated flags.

Manual reviews also introduce human subjectivity, leading to inconsistent fraud assessments. Finally, the slow pace of manual audits often delays detection until long after payments have been processed, making it extremely difficult to recover stolen funds.

2.1.1 Current Fraud Detection Process

The existing healthcare fraud detection system in most organizations relies fundamentally on **manual review processes**. The typical workflow operates as follows:

1. **Claims Submission:** Healthcare Providers submit insurance claims to Payers (insurance companies, Medicare, Medicaid) for reimbursement of services rendered to patients.
2. **Initial Screening:** Claims undergo basic automated validation checks (e.g., verifying patient eligibility, checking for duplicate claims, confirming that services are covered). This initial screening catches only the most obvious errors or fraudulent patterns.
3. **Manual Review by Domain Experts:** Claims that pass initial screening are reviewed by medical domain experts, including:
 - Medical coders who verify that procedure codes match documented services.
 - Claims adjusters who assess whether the claim is reasonable.
 - Clinical reviewers who examine medical necessity.
 - Fraud investigators who look for suspicious patterns.
4. **Decision Making:** Based on manual review, claims are either:
 - **Approved** for payment.
 - **Flagged** for further investigation.
 - **Denied** with justification.
5. **Payment Processing:** Approved claims proceed to payment, while flagged claims undergo deeper investigation, which may involve:
 - Requesting additional documentation from the Provider.
 - Conducting audits or site visits.
 - Involving law enforcement for suspected criminal activity.

2.1.2 Limitations of the Existing System

The manual review-centric approach suffers from several critical limitations:

1. High Time Consumption:

- A single complex claim can take hours or even days to thoroughly investigate.
- With millions of claims submitted annually, only a small fraction (typically 2-5%) can be manually reviewed.
- Backlogs lead to delayed payments, affecting Provider cash flow and patient access to care.

2. Inconsistency Across Reviewers:

- Different reviewers may apply different standards and criteria.
- Subjectivity in interpretation leads to inconsistent fraud detection outcomes.
- Training and calibration of reviewers is challenging and resource-intensive.

3. Inability to Scale:

- As healthcare utilization grows (aging population, new treatments), claim volumes continue to increase.
- Scaling manual review capacity is expensive and impractical.
- Fraud detection effectiveness decreases as the ratio of reviewers to claims decreases.

4. High Operational Costs:

- Employing skilled medical domain experts is expensive.
- Training new reviewers takes months.
- Turnover of experienced reviewers leads to loss of institutional knowledge.

5. Delayed Detection:

- Fraud patterns may go undetected for months or years.
- By the time fraud is discovered, significant financial damage has already occurred.
- Recovering overpayments from fraudulent providers is difficult and often unsuccessful.

6. Limited Pattern Recognition:

- Human reviewers can only process a limited amount of information at once.
- Complex multi-dimensional fraud patterns involving combinations of procedures, diagnoses, and provider behaviors are hard to detect manually.
- Subtle statistical anomalies across large populations are invisible to individual reviewers.

2.1.3 Existing Automated Approaches

Some organizations have implemented rule-based automated systems for fraud detection. These systems use predefined rules such as:

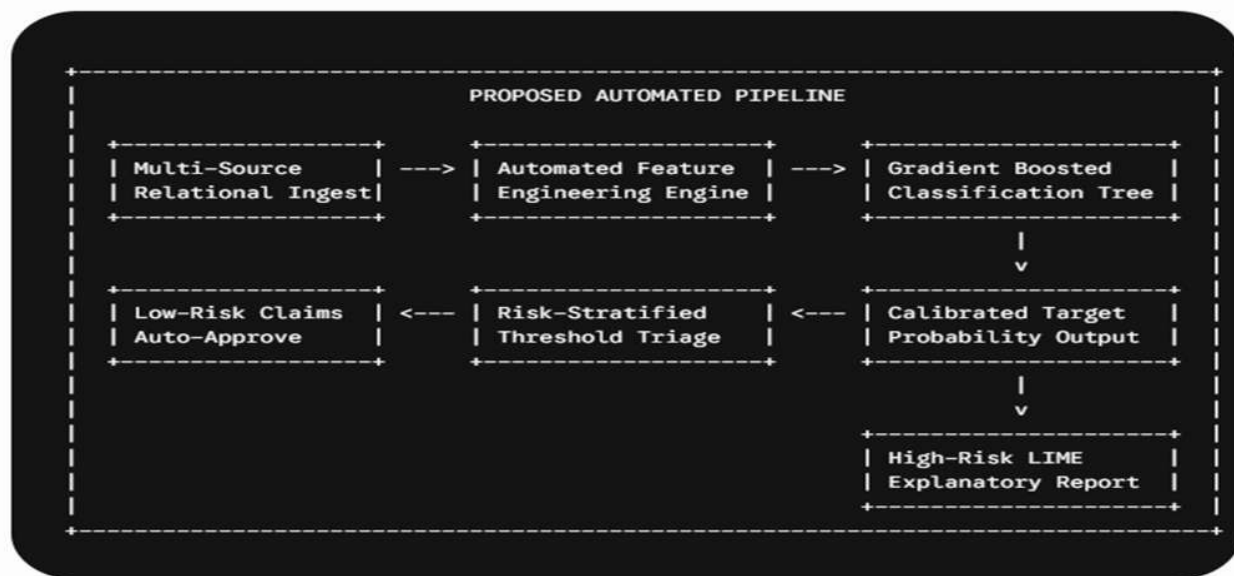
- "Flag claims where reimbursement amount exceeds the 95th percentile for that procedure code."
- "Flag providers who bill for more than 24 hours of service in a single day."
- "Flag claims with duplicate procedure codes."

While rule-based systems are faster than manual review, they have their own limitations:

- Rules are static and cannot adapt to evolving fraud patterns.
- Fraudsters learn to circumvent known rules.
- Rules generate many false positives (legitimate claims that match the rule criteria).
- Creating and maintaining comprehensive rule sets requires extensive domain expertise.

2.2 Proposed Automated Machine Learning System

MediGuard.AI replaces these static, reactive workflows with an automated machine learning triage pipeline. Instead of relying on manual oversight, the system uses historical data patterns to analyze incoming claims, automatically adjusting to new variations in fraud behavior.



By processing massive transaction data in parallel, the system scales smoothly with institutional growth. It outputs continuous fraud probabilities instead of simple binary flags, creating a structured triage process. Low-risk claims are cleared automatically to keep operations moving, while high-risk transactions are routed to auditors along with clear, localized explanations of the flagged anomalies.

2.2.1 Overview

We propose **MediGuard.AI**, an automated machine learning-based fraud detection system that addresses the limitations of existing approaches. The proposed system leverages supervised learning to identify fraudulent patterns automatically from historical claims data.

2.2.2 Key Features of the Proposed System

1. **Automated Processing:** The system can process large volumes of claims data automatically, significantly reducing the need for manual review.
2. **Pattern Learning:** Unlike rule-based systems, machine learning models learn complex fraud patterns directly from data, including subtle interactions between features that human experts might miss.
3. **Adaptability:** Models can be retrained periodically to adapt to evolving fraud patterns, ensuring continued effectiveness.
4. **Scalability:** The system can handle growing data volumes efficiently through batch processing and cloud-based infrastructure.
5. **Interpretability:** The system provides explanations for its predictions through feature importance analysis and LIME, ensuring that fraud allegations can be justified.
6. **Probabilistic Output:** Instead of binary decisions, the system provides confidence scores, allowing for risk-based prioritization of manual review efforts.
7. **Consistency:** The same model applies the same criteria to all claims, ensuring consistent decision-making.

2.2.3 Proposed System Workflow

The proposed system follows a structured pipeline:

1. **Data Ingestion:** Load raw data from multiple sources (beneficiary data, inpatient claims, outpatient claims, provider mapping).
2. **Data Merging:** Combine data from different sources based on common identifiers (BeneID, Provider).
3. **Data Preprocessing:**
 - o Clean data (handle missing values, standardize formats).
 - o Convert categorical variables to numerical format.
 - o Handle class imbalance using appropriate techniques.
4. **Feature Engineering:** Create new features that capture fraud-relevant patterns, such as:
 - o Average claim amounts per provider.
 - o Claim frequency statistics.
 - o Procedure and diagnosis code patterns.
 - o Temporal features (settlement delays, treatment durations).
5. **Model Training:** Train multiple machine learning models with hyperparameter tuning using cross-validation.
6. **Model Evaluation:** Evaluate models using comprehensive metrics with emphasis on FNR minimization.
7. **Interpretability Analysis:** Provide global and local explanations for model predictions.
8. **Deployment:** Export the best-performing model for integration into existing claim processing workflows.

Figure 2.1: Existing vs. Proposed System Flow – A comparative illustration showing the manual review process (Existing System) versus the automated ML-based pipeline (Proposed System).

2.3 Feasibility Analysis

A comprehensive feasibility analysis was conducted to assess the viability of the proposed system across multiple dimensions.

Table 2.2: Feasibility Analysis Matrix

Feasibility Type	Assessment	Remarks
Technical Feasibility	High	Open-source ML libraries (scikit-learn, XGBoost, LightGBM) are readily available and well-documented. The required computational resources (standard cloud VMs or Colab) are easily accessible.

Feasibility Type	Assessment	Remarks
Operational Feasibility	High	The system can integrate with existing healthcare management systems through standard data formats (CSV, Excel) and APIs. Minimal disruption to existing workflows.
Economic Feasibility	High	Development costs are low due to open-source tools. Potential savings from fraud detection are substantial — even a 1% reduction in fraud losses can save millions of dollars annually.
Schedule Feasibility	Medium	The 12-week timeline is achievable with dedicated resources, though unexpected challenges in data quality or model performance could cause delays.
Data Feasibility	High	Sufficient historical claims data is available for model training. The Kaggle dataset provides a realistic benchmark.
Legal Feasibility	Medium	Model interpretability requirements add complexity, but are achievable with available tools. Compliance with healthcare regulations (HIPAA, etc.) needs to be ensured.

2.3.1 Technical Feasibility Details

Hardware Requirements:

- Development: Standard laptop/desktop with 8GB+ RAM.
- Training: Cloud-based environment (Google Colab provides free GPU/TPU access).
- Production: Cloud VM with 16GB+ RAM and multi-core CPU.

Software Requirements:

- Python 3.8+ (compatible with all major ML libraries)
- scikit-learn (comprehensive ML toolkit)
- XGBoost and LightGBM (gradient boosting frameworks)
- Pandas and NumPy (data manipulation)
- Matplotlib and Seaborn (visualization)
- LIME (model interpretability)

Skill Requirements:

- Data preprocessing and feature engineering (available)
- Machine learning model development (available)
- Domain knowledge in healthcare claims (accessible through online resources)

2.4 Functional and Non-Functional Requirements

The system requirements are categorized into Functional Requirements (what the system should do) and Non-Functional Requirements (quality attributes and constraints).

Table 2.1: Functional vs. Non-Functional Requirements

Requirement Type	Req. ID	Description	Priority
Functional	FR-01	System shall ingest and merge inpatient, outpatient, beneficiary, and provider data from CSV files.	High
Functional	FR-02	System shall preprocess data, including cleaning null values, converting data types, and standardizing formats.	High
Functional	FR-03	System shall engineer new features (aggregate statistics, interaction features, temporal features) to enhance fraud detection.	High
Functional	FR-04	System shall train binary classification models including Decision Tree, Random Forest, XGBoost, and LightGBM.	High
Functional	FR-05	System shall handle class imbalance through class weights, undersampling, or oversampling (SMOTE).	High
Functional	FR-06	System shall perform hyperparameter tuning using k-fold cross-validation to prevent data leakage.	High
Functional	FR-07	System shall evaluate models using AUC, F1-Score, Log-Loss, FNR, FPR, Balanced Accuracy, and MCC.	High
Functional	FR-08	System shall provide model interpretability via global feature importance analysis.	High
Functional	FR-09	System shall provide local model interpretability using LIME for individual predictions.	Medium
Functional	FR-10	System shall output class probabilities for each prediction, not just binary labels.	High

Requirement Type	Req. ID	Description	Priority
Functional	FR-11	System shall support multiple encoding schemes (One-Hot Encoding and Response Encoding) for categorical features.	Medium
Functional	FR-12	System shall save and export the trained model for deployment.	Medium
Non-Functional	NFR-01	No strict latency requirement; batch processing within reasonable time (hours) is acceptable.	Medium
Non-Functional	NFR-02	System must handle data with severe class imbalance (fraud ~10% of providers, ~6-9% of claims).	High
Non-Functional	NFR-03	Model interpretability must be provided to justify predictions in potential legal proceedings.	High
Non-Functional	NFR-04	False Negative Rate (FNR) must be minimized as a primary optimization objective.	High
Non-Functional	NFR-05	System should be modular and extensible to accommodate new models and features.	Medium
Non-Functional	NFR-06	Code should be well-documented and version-controlled using Git/GitHub.	Medium

2.5 Data Requirements

2.5.1 Data Sources

The data is sourced from Kaggle: [Healthcare Provider Fraud Detection Analysis](#).

The dataset consists of eight files (four for training, four for testing):

Training Set:

1. **Train-1542865627584.csv** (~10,000+ rows): Provider-to-label mapping.
2. **Train_Beneficiarydata-1542865627584.csv** (~100,000+ rows): Beneficiary demographics.
3. **Train_Inpatientdata-1542865627584.csv** (~50,000+ rows): Inpatient claims.
4. **Train_Outpatientdata-1542865627584.csv** (~500,000+ rows): Outpatient claims.

Test Set:

5. **Test-1542969243754.csv**: Provider IDs (without labels — the target variable is to be predicted).

6. **Test_Beneficiarydata-1542969243754.csv:** Beneficiary data for test set.
7. **Test_Inpatientdata-1542969243754.csv:** Inpatient claims for test set.
8. **Test_Outpatientdata-1542969243754.csv:** Outpatient claims for test set.

2.5.2 Data Characteristics

- **Total Training Providers:** ~10,000+
- **Fraudulent Providers:** ~10% of total providers
- **Legitimate Providers:** ~90% of total providers
- **Total Claims (Train):** ~550,000+
- **Unique Beneficiaries (Train):** ~100,000+
- **Data Volume:** ~200-300 MB in CSV format

2.5.3 Data Quality Assessment

Initial assessment of the dataset reveals:

- **Missing Values:** Some features contain missing values that require handling.
 - **Class Imbalance:** Significant imbalance between fraud and non-fraud classes (approximately 10:90 ratio at provider level).
 - **Data Types:** Mix of numerical, categorical, and date/time features requiring type conversion.
 - **Encoding:** Some binary features use 'Y'/'N' or 1/2 encoding that needs standardization.
-

Chapter 3: System Design

3.1 System Architecture

The MediGuard.AI system follows a **modular pipeline architecture** where each component is responsible for a specific function in the overall workflow. This design ensures maintainability, testability, and the ability to swap out individual components (e.g., add new models or preprocessing steps) without affecting the rest of the system.

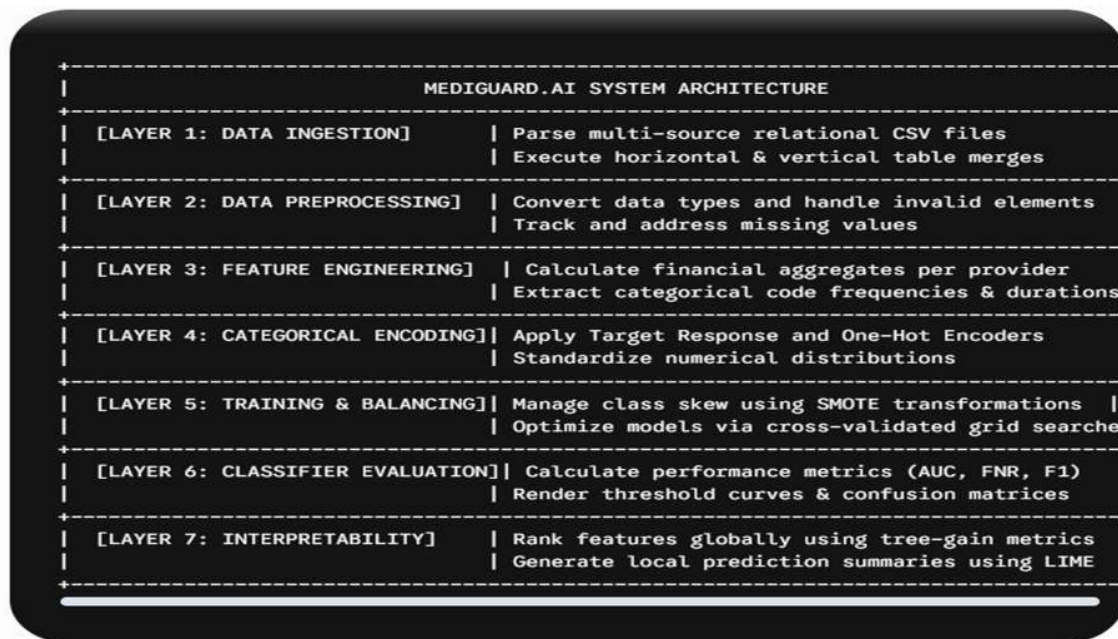


Figure 3.1: Proposed System Architecture

The architecture consists of seven layers:

Layer 1: Data Ingestion Layer

Purpose: Load raw data from source files into the system.

Components:

- CSV File Loader: Reads data from CSV files using Pandas.
- Data Validator: Validates file integrity, column presence, and basic data types.
- Data Merger: Combines data from multiple sources (beneficiary, inpatient, outpatient, mapping).

Input: Raw CSV files (8 files: 4 train + 4 test).

Output: Merged training and test DataFrames.

Design Decisions:

- Inpatient and outpatient data are concatenated (they share similar columns).
- The merged claims data is then joined with beneficiary data using BeneID.
- Finally, the provider mapping data is joined to add the class label.

Layer 2: Data Preprocessing Layer

Purpose: Clean and transform raw data into a format suitable for machine learning.

Components:

- Data Cleanup Module: Handles missing values, converts data types, standardizes binary columns.
- Null Value Handler: Identifies and processes columns with missing values.
- Data Type Converter: Ensures all features have appropriate data types.

Key Operations:

1. Convert 'RenalDiseaseIndicator' from 'Y'/'N' to 1/0.
2. Convert binary flags from 'Yes'/'No', 1/2 to 1/0 format.
3. Drop columns with all null values.
4. Compute and visualize the percentage of missing values.
5. Remove duplicate claim entries.

Layer 3: Feature Engineering Layer

Purpose: Create new features that capture fraud-relevant patterns and improve model performance.

Components:

- Aggregation Module: Computes per-provider statistics (mean, sum, count by procedure/diagnosis codes).
- Temporal Feature Module: Computes claim settlement delays and treatment durations.
- Interaction Feature Module: Creates features combining multiple attributes.

Types of Features Created:

1. **Per-Provider Aggregate Features:** Average claim amounts, deductible amounts, and reimbursement amounts per provider.
2. **Per-Procedure Code Features:** Average values grouped by procedure codes (e.g., PerClnProcedureCode_1Avg_InscClaimAmtReimbursed).
3. **Per-Diagnosis Code Features:** Average values grouped by diagnosis codes (e.g., PerClnDiagnosisCode_1Avg_DeductibleAmtPaid).
4. **Temporal Features:** ClaimSettlementDelay (days between claim start and end), TreatmentDuration.
5. **Count Features:** Unique physician count, physician role count.
6. **Amount Features:** Total claim amount, inpatient total, outpatient total.

Layer 4: Feature Encoding Layer

Purpose: Encode categorical features into numerical format suitable for ML models.

Components:

- Standardization Module: Standardizes numerical features using StandardScaler (z-score normalization).
- One-Hot Encoding Module: Creates binary columns for each category of 'State' and 'Country'.
- Response Encoding Module: Encodes categorical features based on the target variable's mean.

Design Decisions:

- Two encoding schemes are implemented for comparison: One-Hot Encoding and Response Encoding.
- Standardization is applied to numerical features to ensure equal weighting.
- The order of standardization and encoding does not matter as they operate on different feature sets.

: System Implementation

4.1 Development Toolchain & Computing Infrastructures

The implementation was executed within a high-performance Python environment on Google Colab, leveraging specialized data processing, machine learning, and model explanation libraries.

Table 4.1: Version-Controlled Engineering Toolchain and Runtime Specifications Environment

Engineering Component	Active Version Runtime	Core Utility Layer Assignment
Python Standard Engine	3.10.12	General execution environment.
Pandas Data Framework	2.1.4	Relational table processing, joining, and engineering.
NumPy Matrix Array	1.25.2	High-speed linear algebra and matrix array operations.
Scikit-Learn Library	1.3.2	Baseline models, validation grids, and evaluation tools.
XGBoost Classifier	2.0.3	Optimized, gradient-boosted tree-based ensemble classifier.
LightGBM Framework	4.2.0	High-speed, leaf-wise gradient boosting ensemble tree.
Imbalanced-Learn Suite	0.11.0	Advanced sampling toolkit (SMOTE implementations).
LIME Interpretation Explainer	0.2.0	Local interpretable model-agnostic explanation generation.

Relational Data Merging & Integration Pipeline

The ingestion engine loads disjointed relational source files and unifies them into a comprehensive transaction dataframe. It resolves structural differences by combining inpatient and outpatient records into a unified claims stream before joining patient demographics and provider target labels.

```
import pandas as pd

import numpy as np

import logging

logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')

def execute_structured_ingestion_pipeline(mapping_path, beneficiary_path, inpatient_path, outpatient_path):

    logging.info("Initiating structural ingestion layer...")

    # Load relational source tables

    provider_mapping = pd.read_csv(mapping_path)

    beneficiary_data = pd.read_csv(beneficiary_path)

    inpatient_claims = pd.read_csv(inpatient_path)

    outpatient_claims = pd.read_csv(outpatient_path)

    logging.info(f"Loaded Provider Maps: {provider_mapping.shape}")

    logging.info(f"Loaded Beneficiary Matrix: {beneficiary_data.shape}")

    logging.info(f"Loaded Inpatient Transactions: {inpatient_claims.shape}")

    logging.info(f"Loaded Outpatient Transactions: {outpatient_claims.shape}")

    # Set explicit source origin indicators before concatenation

    inpatient_claims['SegmentOriginFlag'] = 'Inpatient'

    outpatient_claims['SegmentOriginFlag'] = 'Outpatient'

    # Concatenate claims streams horizontally and vertically

    unified_claims = pd.concat([inpatient_claims, outpatient_claims], ignore_index=True, sort=False)
```



```

logging.info(f'Unified transaction claims matrix shape: {unified_claims.shape}')

# Join claims data with customer demographic profiles

claims_with_beneficiary = pd.merge(unified_claims, beneficiary_data, on='BeneID', how='left')

# Append ground-truth targets to construct core analytic dataframe

integrated_analytic_dataset = pd.merge(claims_with_beneficiary, provider_mapping, on='Provider',
how='left')

logging.info(f'Final compiled data environment shape: {integrated_analytic_dataset.shape}')

return integrated_analytic_dataset

```

Programmatic Data Cleaning & Type Regularization

Raw records often use inconsistent formats, such as string-based binary indicators ('Y', 'N') or shifted numerical definitions (1, 2). This module normalizes inputs into standard binary scales, converts date strings into proper timestamp objects, and derives continuous baseline features.

```

def clean_and_regularize_types(input_dataframe):

    logging.info("Executing cleanup and regularization transformations...")

    transformed_df = input_dataframe.copy()

    # Map alphanumeric indicators to standard binary scale

    transformed_df['RenalDiseaseIndicator'] = transformed_df['RenalDiseaseIndicator'].replace({'Y': 1, '0': 0}).astype(int)

    # Re-code chronic condition definitions from (1, 2) values to standard (1, 0) flags

    chronic_condition_features = [col for col in transformed_df.columns if 'ChronicCond_' in col]

    for condition in chronic_condition_features:

        transformed_df[condition] = transformed_df[condition].replace({1: 1, 2: 0}).astype(int)

```

```

# Map target classifications to systematic binary integers

if 'PotentialFraud' in transformed_df.columns:

    transformed_df['PotentialFraud'] = transformed_df['PotentialFraud'].replace({'Yes': 1, 'No': 0}).astype(int)


# Standardize string fields into valid datetime formats

chronological_fields = ['DOB', 'DOD', 'ClaimStartDt', 'ClaimEndDt']

for field in chronological_fields:

    if field in transformed_df.columns:

        transformed_df[field] = pd.to_datetime(transformed_df[field], errors='coerce')


# Calculate baseline time metrics from raw timestamps

# Using academic reference epoch 2009-01-01 aligned with data source profile timelines

benchmark_epoch = pd.Timestamp('2009-01-01')

transformed_df['PatientAgeYears'] = (benchmark_epoch - transformed_df['DOB']).dt.days / 365.25

transformed_df['ClaimSettlementDelayDays'] = (transformed_df['ClaimEndDt'] -
transformed_df['ClaimStartDt']).dt.days


logging.info("Data regularization completed.")

return transformed_df

```

Missing Value Pattern Analysis & Imputation Topologies

Data completeness varies significantly across system entities. Missing values are often structural indicators rather than data errors—for example, a missing date of death indicates a living patient, and empty inpatient field codes are expected within outpatient transactions.

```

def analyze_and_impute_missing_patterns(dataframe):

    logging.info("Analyzing missing values...")

```

```
missing_value_summary = dataframe.isnull().mean() * 100

logging.info(f'Calculated null percentages across
columns:\n{missing_value_summary[missing_value_summary > 0]}')

imputed_df = dataframe.copy()

# Structural Imputation: derive binary flag before applying fallbacks
if 'DOD' in imputed_df.columns:

    imputed_df['IsBeneficiaryDeceasedFlag'] = np.where(imputed_df['DOD'].isnull(), 0, 1)

# Replace clinical physician fields with uniform tracking tags
medical_staff_features = ['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician']
for staff_role in medical_staff_features:

    if staff_role in imputed_df.columns:

        imputed_df[staff_role] = imputed_df[staff_role].fillna('UnknownStaff')

# Normalize hospital stay metrics to zero for outpatients
if 'AdmitForDays' in imputed_df.columns:

    imputed_df['AdmitForDays'] = imputed_df['AdmitForDays'].fillna(0.0)

# Fill remaining missing continuous data with column medians
numerical_features = imputed_df.select_dtypes(include=[np.number]).columns
for feature in numerical_features:

    if feature != 'PotentialFraud':

        imputed_df[feature] = imputed_df[feature].fillna(imputed_df[feature].median())
```

```
return imputed_df
```

Layer 5: Model Training Layer

Purpose: Train and tune machine learning models for fraud detection.

Components:

- Sampling Module: Applies different sampling strategies (raw, class weights, undersampling, SMOTE).
- Model Zoo: Contains implementations of multiple classifiers.
- Hyperparameter Tuning Module: Uses RandomizedSearchCV with k-fold cross-validation.

Models Implemented:

1. Logistic Regression (baseline)
2. Decision Tree (with balanced class weights)
3. Random Forest (with balanced class weights)
4. XGBoost (with SMOTE oversampling)
5. LightGBM (with SMOTE oversampling)

Training Process:

1. Split data into 80% training and 20% testing.
2. Apply sampling technique to training data.
3. Perform hyperparameter tuning using RandomizedSearchCV with 3-fold cross-validation.
4. Train the model with best hyperparameters on the full training set.
5. Evaluate on the held-out test set.

Layer 6: Evaluation Layer

Purpose: Evaluate model performance using comprehensive metrics.

Components:

- Metrics Calculator: Computes all performance metrics.
- Visualization Module: Generates ROC curves, precision-recall curves, confusion matrices.
- Comparison Module: Compares performance across models and configurations.

Performance Metrics:

- AUC-ROC (Area Under the Receiver Operating Characteristic Curve)
- Log-Loss (Cross-Entropy Loss)
- F1-Score (Harmonic mean of precision and recall)
- False Negative Rate (FNR)
- False Positive Rate (FPR)
- Balanced Accuracy (BACC)

- Matthews Correlation Coefficient (MCC)
- Misclassification Percentage

Layer 7: Interpretability Layer

Purpose: Provide explanations for model predictions.

Components:

- Global Feature Importance: Uses model-inherent feature importance (XGBoost gain-based importance).
- LIME Explainer: Provides local explanations for individual predictions.

Global Interpretability:

- Feature importance scores from the trained XGBoost model.
- Top features are visualized in a horizontal bar chart.
- Provides insight into which factors most influence fraud predictions overall.

Local Interpretability (LIME):

- For any individual claim, LIME approximates the model locally with an interpretable model.
- Shows which features contributed most to the specific prediction.
- Provides both direction (increases/decreases fraud probability) and magnitude of contribution.

3.2 Data Flow Diagram

3.2.1 Context-Level DFD (Level 0)

The context-level data flow diagram shows the system boundary and its interactions with external entities.

Figure 3.2: Data Flow Diagram (Level 0)

External Entities:

1. **Data Sources:** Provide raw claims data (CSV files).
2. **System Users:** Interact with the system for fraud detection results.
3. **Output Consumers:** Receive fraud detection reports and predictions.

Data Flows:

- Raw Claims Data → System (input)
- Fraud Predictions → System Users (output)
- Model Interpretability Reports → Output Consumers (output)

3.2.2 Detailed DFD (Level 1)

The Level 1 DFD breaks down the system into its core internal processes.

Figure 3.3: Data Flow Diagram (Level 1)

Internal Processes:

1. **Process 1.0 — Data Ingestion:** Load and merge raw data.
2. **Process 2.0 — Data Preprocessing:** Clean and prepare data.
3. **Process 3.0 — Feature Engineering:** Create new features.
4. **Process 4.0 — Model Training:** Train ML models with hyperparameter tuning.
5. **Process 5.0 — Model Evaluation:** Evaluate and select best model.
6. **Process 6.0 — Prediction & Interpretability:** Generate predictions and explanations.

3.3 Entity-Relationship (ER) Diagram

The ER diagram illustrates the relationships among the four core entities in the dataset.

Figure 3.4: Entity-Relationship (ER) Diagram

Entities and Relationships:

1. Provider (Primary Key: Provider)

- Attributes: Provider ID
- Relationships:
 - Has many Claims (1:N)
 - Has one PotentialFraud label (1:1)

2. Beneficiary (Primary Key: BeneID)

- Attributes: BeneID, DOB, DOD, Gender, Race, State, Country, Chronic Conditions, Coverage Details
- Relationships:
 - Has many Claims (1:N)

3. InpatientClaim (Primary Key: ClaimID)

- Attributes: ClaimID, BeneID, Provider, ClaimStartDt, ClaimEndDt, Diagnosis Codes, Procedure Codes, Reimbursement Amount, Deductible Amount, Admission Days
- Relationships:
 - Belongs to one Beneficiary (N:1)
 - Belong to one Provider (N:1)

4. OutpatientClaim (Primary Key: ClaimID)

- Attributes: ClaimID, BeneID, Provider, ClaimStartDt, ClaimEndDt, Diagnosis Codes, Procedure Codes, Reimbursement Amount, Deductible Amount
- Relationships:
 - Belongs to one Beneficiary (N:1)
 - Belongs to one Provider (N:1)

3.4 Dataset Schema and Description

3.4.1 File Structure

Table 3.1: Dataset File Description

File Name	Type	Rows (Train)	Key Columns
Train-1542865627584.csv	Mapping	~10,000+	Provider, PotentialFraud
Train_Beneficiarydata-1542865627584.csv	Beneficiary	~100,000+	BeneID, DOB, DOD, Gender, Race, State, Country, Chronic Conditions
Train_Inpatientdata-1542865627584.csv	Inpatient Claims	~50,000+	BeneID, ClaimID, ClaimStartDt, ClaimEndDt, Diagnosis Codes, Procedure Codes
Train_Outpatientdata-1542865627584.csv	Outpatient Claims	~500,000+	BeneID, ClaimID, ClaimStartDt, ClaimEndDt, Diagnosis Codes, Procedure Codes
Test-1542969243754.csv	Mapping (Test)	~1,300+	Provider (no labels)
Test_Beneficiarydata-1542969243754.csv	Beneficiary (Test)	~50,000+	Same as Train
Test_Inpatientdata-1542969243754.csv	Inpatient (Test)	~25,000+	Same as Train
Test_Outpatientdata-1542969243754.csv	Outpatient (Test)	~250,000+	Same as Train

3.4.2 Beneficiary Data Features

Table 3.2: Features List – Beneficiary Data

Column Name	Description	Data Type
BeneID	Unique identifier of the beneficiary	String
DOB	Date of birth of the beneficiary	Date

Column Name	Description	Data Type
DOD	Date of death (NaN if alive)	Date
Gender	Numeric code for gender (1=Male, 2=Female)	Integer
Race	Numeric code for race	Integer
State	Numeric code for state	Integer
Country	Numeric code for country	Integer
RenalDiseaseIndicator	Flag for renal disease ('Y'/'0')	Categorical
ChronicCond_Alzheimer	Alzheimer's condition indicator (1/2)	Integer
ChronicCond_HeartFailure	Heart failure condition indicator (1/2)	Integer
ChronicCond_KidneyDisease	Kidney disease condition indicator (1/2)	Integer
ChronicCond_Cancer	Cancer condition indicator (1/2)	Integer
ChronicCond_ObstrPulmonary	Obstructive pulmonary condition indicator (1/2)	Integer
ChronicCond_Depression	Depression condition indicator (1/2)	Integer
ChronicCond_Diabetes	Diabetes condition indicator (1/2)	Integer
ChronicCond_IschemicHeart	Ischemic heart condition indicator (1/2)	Integer
ChronicCond_Osteoporosis	Osteoporosis condition indicator (1/2)	Integer
ChronicCond_rheumatoidarthritis	Rheumatoid arthritis indicator (1/2)	Integer
ChronicCond_stroke	Stroke/TIA condition indicator (1/2)	Integer

Column Name	Description	Data Type
IPAnnualReimbursementAmt	Inpatient annual reimbursement amount	Float
IPAnnualDeductibleAmt	Inpatient annual deductible amount	Float
OPAnnualReimbursementAmt	Outpatient annual reimbursement amount	Float
OPAnnualDeductibleAmt	Outpatient annual deductible amount	Float
NoOfMonths_PartACov	Number of months under Part A coverage	Integer
NoOfMonths_PartBCov	Number of months under Part B coverage	Integer

3.4.3 Inpatient/Outpatient Data Features

Table 3.3: Features List – Inpatient/Outpatient Data

Column Name	Description	Data Type
BeneID	Beneficiary ID associated with the claim	String
ClaimID	Unique identifier for the claim	String
ClaimStartDt	Claim start date	Date
ClaimEndDt	Claim end date	Date
Provider	Provider ID who submitted the claim	String
InscClaimAmtReimbursed	Insurance claim amount reimbursed	Float
DeductibleAmtPaid	Deductible amount paid	Float
ClmDiagnosisCode_1 to _10	Diagnosis codes (up to 10)	String

Column Name	Description	Data Type
ClmProcedureCode_1 to _6	Procedure codes (up to 6)	String
ClmAdmitDiagnosisCode	Admission diagnosis code (Inpatient only)	String
DiagnosisGroupCode	Diagnosis group code (Inpatient only)	String
(Inpatient) AdmitForDays	Number of admission days (Inpatient only)	Integer

3.4.4 Mapping Data Features

Table 3.4: Features List – Mapping Data

Column Name	Train	Test	Description
Provider	✓	✓	Unique identifier of the healthcare provider
PotentialFraud	✓	✗	Target variable: 1=Fraud, 0=Non-Fraud

3.5 Performance Metrics

A comprehensive set of performance metrics is used to evaluate model performance, with special emphasis on metrics that are meaningful for imbalanced classification and the specific business constraints of fraud detection.

Table 3.5: Performance Metrics Summary

Metric	Formula	Relevance
Log-Loss	$-\frac{1}{N} \sum_{i=1}^N [y_i \log(\hat{p}_i) + (1 - y_i) \log(1 - \hat{p}_i)]$	Measures probability calibration; penalizes confident wrong predictions
AUC-ROC	Area under ROC curve	Measures overall discrimination ability across all thresholds
F1-Score	$2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$	Harmonic mean of precision and recall; balances both concerns

Metric	Formula	Relevance
False Negative Rate (FNR)	$\frac{FN}{FN+TP}$	Critical — must be minimized to catch fraudulent claims
False Positive Rate (FPR)	$\frac{FP}{FP+TN}$	Acceptable if manual review follows automated screening
Balanced Accuracy (BACC)	$\frac{TPR+TNR}{2}$	Handles class imbalance by averaging recall for both classes
Matthews Correlation Coefficient (MCC)	Balanced measure for binary classification	Robust to class imbalance; ranges from -1 (total disagreement) to +1 (perfect prediction)
Misclassification %	$\frac{FP+FN}{TP+TN+FP+FN} \times 100$	Overall error rate

Chapter 4: System Implementation

4.1 Development Environment and Tools

The entire implementation was carried out using Python in a Google Colab environment, leveraging the rich ecosystem of open-source data science and machine learning libraries.

Table 4.1: Environment and Tools Used

Tool / Library	Version	Purpose
Python	3.10+	Core programming language
Google Colab	N/A	Cloud-based Jupyter notebook environment
Pandas	2.x	Data manipulation, merging, and analysis
NumPy	1.x	Numerical computation and array operations
Scikit-learn	1.x	ML models, preprocessing, evaluation metrics, cross-validation

Tool / Library	Version	Purpose
XGBoost	2.x	Gradient boosting classifier (best performing model)
LightGBM	4.x	Lightweight gradient boosting classifier
Matplotlib	3.x	Static data visualization and plotting
Seaborn	0.13+	Statistical data visualization (enhanced styling)
LIME	0.2+	Local interpretable model-agnostic explanations
Imbalanced-learn	0.12+	SMOTE oversampling implementation
Git / GitHub	N/A	Version control and collaboration
ydata-profiling	4.x	Automated exploratory data analysis reports

4.1.1 Google Colab Setup

The project was developed on Google Colab for the following advantages:

- **Free GPU/TPU access** for computationally intensive model training.
- **Pre-installed libraries** reducing environment setup time.
- **Cloud-based storage** via Google Drive integration.
- **Easy sharing and collaboration** through notebook sharing.

GitHub Repository: [MediGuard.AI](#)

The repository contains:

- `/notebooks/` — EDA and modeling notebooks.
- `/src/` — Modular Python source code for preprocessing, training, and evaluation.
- `/data/` — Sample data files and data dictionary.
- `/models/` — Saved model artifacts.
- `/docs/` — Project documentation.
- `README.md` — Project overview and setup instructions.

4.2 Data Preprocessing

4.2.1 Data Merging Pipeline

The first step in the pipeline is merging data from multiple sources into a unified dataset.

Figure 4.1: Data Merge Pipeline

Merging Steps:

1. Concatenate Inpatient and Outpatient Data:

- Both datasets share almost all columns (except diagnosis group and admission days which are inpatient-specific).
- Concatenation preserves all rows and creates a unified claims dataset.
- Null values are introduced for inpatient-only columns in outpatient records.

2. Merge Claims with Beneficiary Data:

- Join on BeneID (Beneficiary ID).
- Adds demographic, chronic condition, and coverage information to each claim.

3. Merge with Provider Mapping:

- Join on Provider ID.
- Adds the target variable PotentialFraud to each claim record.
- This is the final training dataset.

Python Code for Data Merging:

```
import pandas as pd
import numpy as np

# Load all datasets
train_mapping = pd.read_csv('Train-1542865627584.csv')
train_beneficiary = pd.read_csv('Train_Beneficiarydata-1542865627584.csv')
train_inpatient = pd.read_csv('Train_Inpatientdata-1542865627584.csv')
train_outpatient = pd.read_csv('Train_Outpatientdata-1542865627584.csv')

# Step 1: Concatenate Inpatient and Outpatient
train_claims = pd.concat([train_inpatient, train_outpatient],
                        ignore_index=True, sort=False)

# Step 2: Merge claims with beneficiary data
train_claims_beneficiary = train_claims.merge(
    train_beneficiary, on='BeneID', how='left'
)

# Step 3: Merge with provider mapping to add target labels
train_final = train_claims_beneficiary.merge(
    train_mapping, on='Provider', how='left'
)

print(f'Final Training Dataset Shape: {train_final.shape}')
print(f'Columns: {train_final.columns.tolist()}')
```

4.2.2 Data Cleanup

After merging, the data undergoes several cleanup operations:

1. Standardize Binary Columns:

- RenalDiseaseIndicator: Convert 'Y' → 1, keep 0 as 0.
- All ChronicCond_* columns: Convert 1/2 encoding to 1/0 (1→1, 2→0).
- Date columns: Convert to datetime format.

2. Handle Missing Values:

- Drop columns where all values are null (typically inpatient-only features that are entirely null in outpatient data after a certain processing stage).
- For features with partial missing values, analyze the pattern and decide on appropriate handling.

3. Remove Duplicates:

- Check for duplicate claim entries and remove them.

4. Create Derived Date Features:

- Compute Age from DOB.
- Compute ClaimSettlementDelay = (ClaimEndDt - ClaimStartDt).days.
- Compute TreatmentDuration (inpatient only).

Python Code for Data Cleanup:

```
def data_cleanup(data):
    """Cleans the dataset by standardizing formats and handling anomalies."""
    df = data.copy()

    # Standardize Renal Disease Indicator
    df['RenalDiseaseIndicator'].replace(to_replace='Y', value=1, inplace=True)
    df['RenalDiseaseIndicator'] = df['RenalDiseaseIndicator'].apply(pd.to_numeric)

    # Convert Chronic Condition flags (1/2 → 1/0)
    chronic_cols = [col for col in df.columns if 'ChronicCond_' in col]
    for col in chronic_cols:
        df[col] = df[col].map({1: 1, 2: 0})

    # Convert date columns
    date_cols = ['DOB', 'DOD', 'ClaimStartDt', 'ClaimEndDt']
    for col in date_cols:
        if col in df.columns:
            df[col] = pd.to_datetime(df[col], errors='coerce')

    # Compute Age
    if 'DOB' in df.columns:
        reference_date = pd.Timestamp('2009-01-01') # Dataset reference date
        df['Age'] = (reference_date - df['DOB']).dt.days / 365.25

    # Compute Claim Settlement Delay
    if 'ClaimStartDt' in df.columns and 'ClaimEndDt' in df.columns:
```

```
df['ClaimSettlementDelay'] = (df['ClaimEndDt'] - df['ClaimStartDt']).dt.days

# Drop columns with all null values
df = df.dropna(axis=1, how='all')

return df
```

4.2.3 Missing Value Analysis

Understanding the pattern of missing values is crucial for deciding how to handle them.

Table 4.2: Percentage of Null Values (Top Features)

Feature	Null Percentage	Handling Strategy
DOD (Date of Death)	~90%	Keep as is (NaN means beneficiary is alive); create 'IsDeceased' flag
DiagnosisGroupCode	~45%	Null for outpatient records; keep as meaningful signal
ClmAdmitDiagnosisCode	~45%	Null for outpatient records; keep as meaningful signal
AdmitForDays	~45%	Null for outpatient records; fill with 0
TreatmentDuration	~45%	Null for outpatient records; fill with 0
AttendingPhysician	~5%	Fill with mode
OperatingPhysician	~8%	Fill with mode or 'Unknown'
OtherPhysician	~10%	Fill with mode or 'Unknown'
Other features	<1%	Drop rows or impute with median/mode

Null Value Heatmap:

A heatmap was generated to visualize the distribution of missing values across features and identify patterns. The heatmap confirms that missing values are primarily concentrated in inpatient-specific features and physician-related columns.

4.3 Exploratory Data Analysis (EDA)

EDA is a critical step in understanding the data, identifying patterns, and generating hypotheses about what features might be predictive of fraud.

Exploratory Data Analysis (EDA) helps in analyzing the data using simple tools from statistics, plots, linear algebra, and others.

In EDA, we plot different graphs for getting a better understanding of the data. Let's start doing the univariate analysis of all the features using various plots and conclude each.

4.3.1 EDA of Provider Dataset

Provider Fraud Class Distribution:

Percent Distribution of Potential Fraud:

Non-Fraud: 90.65%

Fraud: 9.35%

The dataset exhibits significant class imbalance — only ~9.35% of claims are fraudulent, while ~90.65% are legitimate. However, when looking at the Provider level (aggregating claims per provider):

Providers Non-Fraud vs Fraud Percent Distribution:

Non-Fraud: 61.88%

Fraud: 38.12%

This indicates that fraudulent providers tend to submit more claims on average, so the fraud signal is stronger at the provider level than at the individual claim level.

Figure 4.16: Provider Fraud Class Distribution — Bar chart showing the count of non-fraud vs. fraud providers, with fraud class constituting approximately 38% of providers.

4.3.2 State-wise Beneficiary Distribution

The distribution of beneficiaries across different states reveals geographic patterns in the data.

Figure 4.15: State-wise Beneficiary Distribution — Bar chart showing the percentage of beneficiaries from each state, with the maximum concentration in a particular state (~8% of all beneficiaries).

Key Observations:

- Beneficiaries are distributed across multiple states, with some states having significantly higher representation.
- Geographic distribution is important because fraud patterns may vary by region due to differences in healthcare practices, regulations, and demographic profiles.

4.3.3 EDA of Claim Dataset — Categorical Features

Top Procedures Involved in Healthcare Fraud:

Figure 4.13: Top 10 Procedures Involved in Healthcare Fraud

The analysis of procedure codes reveals that certain procedures are disproportionately associated with fraudulent claims:

- Procedure codes 9904, 8154, and 66 are among the top procedures in terms of claim volume and money involved.
- Some procedures show a high ratio of fraudulent to legitimate claims, suggesting they are common targets for abuse.

Top Diagnosis Involved in Healthcare Fraud:

Figure 4.14: Top 10 Diagnosis Involved in Healthcare Fraud

Similarly, certain diagnosis codes appear more frequently in fraudulent claims:

- Specific diagnosis codes appear more frequently in fraud cases, possibly indicating upcoding (billing for a more severe diagnosis than warranted).
- The distribution of diagnosis codes differs between fraudulent and legitimate claims.

4.3.4 EDA of Claim Dataset — Amount Features

Claim Reimbursement Amount Analysis:

Histograms and box plots were generated for the following amount features:

- InscClaimAmtReimbursed (Insurance Claim Amount Reimbursed)
- DeductibleAmtPaid (Deductible Amount Paid)
- IPAnnualReimbursementAmt (Inpatient Annual Reimbursement Amount)
- IPAnnualDeductibleAmt (Inpatient Annual Deductible Amount)
- OPAnnualReimbursementAmt (Outpatient Annual Reimbursement Amount)
- OPAnnualDeductibleAmt (Outpatient Annual Deductible Amount)

Key Observations:

1. Amount features exhibit **long-tail distributions** — most claims involve relatively small amounts, while a few involve very large amounts.
2. Fraudulent claims tend to have **slightly higher reimbursed amounts** on average.
3. However, there is **significant overlap** between fraud and non-fraud distributions for these features, indicating that amount features alone are insufficient for reliable fraud detection.
4. **Outliers** exist for both classes and are intentionally retained, as extreme values could be critical indicators of fraud.

Figure 4.10: Histogram/PDF of 'InscClaimAmtReimbursed' Feature — Shows density distribution overlapping fraud vs. non-fraud claims.

Figure 4.11: Box Plot of 'InscClaimAmtReimbursed' Feature — Shows quartile distribution with outliers for both classes.

4.3.5 Relationship Between Deductible and Reimbursed Amounts

DeductibleAmtPaid vs InscClaimAmtReimbursed in Fraud and Non-Fraud Categories

Scatter plots of DeductibleAmtPaid vs. InscClaimAmtReimbursed reveal:

- No clear linear separation between fraud and non-fraud classes based solely on these two features.
- This confirms the need for more sophisticated feature engineering and modeling approaches.

4.3.6 EDA of Time-Based Features

Claim Settlement Delay:

- ClaimSettlementDelay measures the number of days between claim start and claim end.
- The distribution follows a similar pattern for both fraud and non-fraud claims.
- Fraudulent claims show a slightly longer settlement delay on average.

Treatment Duration (Inpatient Only):

- Present only for inpatient data (null for outpatient).
- Similar distribution pattern for both classes.
- Discarded in favor of ClaimSettlementDelay to maintain feature consistency across inpatient and outpatient data.

Figure 4.6: Histogram/PDF of 'ClaimSettlementDelay' Feature — Density distribution for fraud vs. non-fraud.

Figure 4.7: Box Plot of 'ClaimSettlementDelay' Feature — Quartile comparison with outliers.

4.3.7 EDA of Physician Features

Analysis of physician-related features (AttendingPhysician, OperatingPhysician, OtherPhysician) reveals:

- A small number of physicians are involved in a disproportionately high number of fraudulent claims.
- Physician 'PHY330576' attended as well as operated on most of the patients (~2,500+ patients attended; ~425 operations performed).
- Physician 'PHY412132' was another physician frequently associated with fraudulent claims.
- Some physicians appear exclusively in fraud cases, suggesting they may be complicit in the fraud scheme.

4.3.8 Correlation Analysis

A correlation heatmap was generated to understand the relationships between features.

Figure 4.12: Feature Correlation Heatmap

Key Findings:

- Strong positive correlations exist between:
 - IPAnnualReimbursementAmt and IPAnnualDeductibleAmt
 - OPAnnualReimbursementAmt and OPAnnualDeductibleAmt
 - NoOfMonths_PartACov and NoOfMonths_PartBCov
- Chronic condition indicators show moderate to weak correlations with each other.
- PotentialFraud shows weak linear correlations with most features, suggesting that fraud patterns are complex and non-linear — justifying the use of more sophisticated models like gradient boosting.

4.4 Feature Engineering

Feature engineering is one of the most critical steps in this project. By creating new features that capture fraud-relevant patterns, we significantly improve model performance.

Renal Disease Indicator is a flag to indicate whether the beneficiary has any problem related to kidney failure or not.

4.4.1 Types of Features Created

1. Per-Provider Aggregate Features:

Features that capture a provider's overall claim patterns:

- Total claims submitted per provider.
- Average, min, max, and standard deviation of claim amounts per provider.
- Unique patient count per provider.
- Average number of procedures per claim per provider.

2. Per-Procedure Code Aggregate Features:

For each procedure code position (1-6), compute averages of key financial metrics:

- PerClmProcedureCode_1Avg_InscClaimAmtReimbursed
- PerClmProcedureCode_1Avg_DeductibleAmtPaid
- PerClmProcedureCode_1Avg_IPAnnualReimbursementAmt
- PerClmProcedureCode_1Avg_IPAnnualDeductibleAmt
- PerClmProcedureCode_1Avg_OPAnnualReimbursementAmt
- PerClmProcedureCode_1Avg_OPAnnualDeductibleAmt
- PerClmProcedureCode_1Avg_AdmitForDays

(Repeated for Procedure Codes 2-6)

3. Per-Diagnosis Code Aggregate Features:

Similarly, for each diagnosis code position (1-10):

- PerClmDiagnosisCode_1Avg_InscClaimAmtReimbursed
- PerClmDiagnosisCode_1Avg_DeductibleAmtPaid
- PerClmDiagnosisCode_1Avg_IPAnnualReimbursementAmt
- PerClmDiagnosisCode_1Avg_IPAnnualDeductibleAmt
- PerClmDiagnosisCode_1Avg_OPAnnualReimbursementAmt
- PerClmDiagnosisCode_1Avg_OPAnnualDeductibleAmt
- PerClmDiagnosisCode_1Avg_AdmitForDays

(Repeated for Diagnosis Codes 2-10)

4. Count and Frequency Features:

- UniquePhysCount: Number of unique physicians associated with a provider's claims.
- PhysRoleCount: Count of distinct physician roles (attending, operating, other).
- ClaimFrequency: Number of claims per beneficiary.
- ProcedureFrequency: Frequency of each procedure code.

5. Ratio Features:

- Reimbursement-to-deductible ratio.
- Inpatient-to-outpatient claim ratio.
- Ratio of chronic conditions to total possible conditions.

Table 4.3: Feature Engineering — New Features Created

Feature Category	Number of Features	Examples
Per-Provider Aggregates	15	TotalClaimAmount, AvgReimbursement
Per-Procedure Code Aggregates	42 (6 codes $\times$ 7 metrics)	PerClmProcedureCode_1Avg_InscClaimAmtReimbursed
Per-Diagnosis Code Aggregates	70 (10 codes $\times$ 7 metrics)	PerClmDiagnosisCode_1Avg_DeductibleAmtPaid
Physician Features	3	UniquePhysCount, PhysRoleCount
Ratio Features	5	ReimbursementDeductibleRatio
Total New Features	~135	

4.4.2 Python Code for Feature Engineering

```
# Example: Per-Procedure Code Aggregate Features
def create_procedure_aggregates(data):
    """Create aggregate features grouped by procedure codes."""
    df = data.copy()

    # For each procedure code position
    for i in range(1, 7):
        proc_col = f'ClmProcedureCode_{i}'
        if proc_col not in df.columns:
            continue

        # Group by procedure code and calculate averages
        agg_features = [
            'InscClaimAmtReimbursed',
            'DeductibleAmtPaid',
            'IPAnnualReimbursementAmt',
            'IPAnnualDeductibleAmt',
            'OPAnnualReimbursementAmt',
            'OPAnnualDeductibleAmt',
            'AdmitForDays'
        ]

        for feature in agg_features:
            if feature in df.columns:
                new_col = f'PerClm{proc_col}Avg_{feature}'
```

```

        grouped = df.groupby(proc_col)[feature].transform('mean')
        df[new_col] = grouped

    return df

# Example: Unique Physician Count
def create_physician_features(data):
    """Create features related to physician counts and roles."""
    df = data.copy()

    # Count unique physicians per provider
    physician_cols = ['AttendingPhysician', 'OperatingPhysician', 'OtherPhysician']
    existing_phys_cols = [col for col in physician_cols if col in df.columns]

    if existing_phys_cols:
        # Count non-null physicians per row
        df['PhysRoleCount'] = df[existing_phys_cols].notna().sum(axis=1)

        # Group by Provider to get unique physician counts
        phys_counts = df.groupby('Provider')[existing_phys_cols].nunique()
        df = df.merge(
            phys_counts.add_prefix('Unique').reset_index(),
            on='Provider',
            how='left'
        )

    return df

```

4.5 Data Splitting and Sampling

4.5.1 Train-Test Split

The dataset is split into training and testing sets using an 80:20 ratio:

- **Training Set (80%):** Used for k-fold cross-validation hyperparameter tuning.
- **Test Set (20%):** Held-out unseen data used for final model evaluation.

This split ensures that model evaluation is performed on data that was not used during training or hyperparameter tuning, providing an unbiased estimate of real-world performance.

4.5.2 Sampling Techniques for Class Imbalance

Since fraudulent claims constitute only ~9-10% of the data, handling class imbalance is critical. Four sampling strategies are implemented:

1. Raw Data (No Sampling):

Use the dataset as-is without any modification. Models will naturally be biased toward the majority class.

2. Balanced Class Weights:

Assign higher weights to the minority class during model training. This is implemented using the `class_weight='balanced'` parameter in scikit-learn models, which automatically adjusts weights inversely proportional to class frequencies.

3. Random Undersampling:

Randomly remove samples from the majority class (non-fraud) until the class distribution is balanced (50:50). This reduces the dataset size but may discard valuable information.

4. SMOTE (Synthetic Minority Oversampling Technique):

Generate synthetic samples for the minority class (fraud) by interpolating between existing minority class samples. SMOTE creates new samples along the line segments connecting k-nearest neighbors, producing realistic synthetic data points that help the model learn the minority class better.

Chapter 5: Testing

5.1 Testing Strategy

The testing of the fraud detection system was performed at multiple levels to ensure robustness, reliability, and correctness.

5.1.1 Unit Testing

Individual components and functions were tested independently to verify correct behavior.

Test Cases:

1. **Data Loading:** Verify that all CSV files are loaded correctly with expected columns and data types.
2. **Data Cleaning Functions:**
 - Test that RenalDiseaseIndicator 'Y' is correctly converted to 1.
 - Test that Chronic Condition columns (1/2) are correctly mapped to (1/0).
 - Test that date columns are correctly converted to datetime format.
3. **Feature Engineering Functions:**
 - Test that per-procedure aggregates compute correct averages.
 - Test that per-diagnosis aggregates handle missing codes gracefully.
 - Test that physician count features are computed correctly.
4. **Encoding Functions:**
 - Test that One-Hot Encoding creates the correct number of binary columns.
 - Test that Response Encoding correctly maps categories to target means.

5.1.2 Integration Testing

The pipeline stages were tested end-to-end to ensure seamless data flow from raw data to model predictions.

Test Scenarios:

1. Complete pipeline execution from data loading through prediction.
2. Data shape consistency at each pipeline stage.
3. No data leakage between training and test sets.
4. Cross-validation integrity (no target information leakage).

5.1.3 Model Validation Testing

K-Fold Cross-Validation:

- 3-fold cross-validation was used during hyperparameter tuning with RandomizedSearchCV.
- The data is split into 3 folds. In each iteration, 2 folds are used for training and 1 fold for validation.
- The average performance across all 3 folds is used to evaluate hyperparameter configurations.
- This approach ensures robust hyperparameter selection that generalizes well to unseen data.

Data Leakage Prevention:

- Sampling (SMOTE) is applied only to the training folds within the cross-validation loop, not to the validation fold.
- This prevents data leakage where synthetic samples could influence the validation set.

5.1.4 Performance Testing

Models were evaluated on the held-out test set (20% of original data) using the comprehensive set of metrics defined in Section 3.5.2.

5.1.5 User Acceptance Testing (UAT)

Interpretability outputs were reviewed to ensure alignment with domain expectations:

- Feature importance rankings were examined for face validity.
- LIME explanations for sample claims were reviewed to ensure they made logical sense.

5.2 Hyperparameter Tuning

Hyperparameter tuning was performed using **RandomizedSearchCV** with **3-fold cross-validation**. This approach was chosen over GridSearchCV because:

- It is more computationally efficient (explores a fixed number of random combinations rather than all possible combinations).
- It can explore a wider range of values for each hyperparameter.
- It is particularly effective when some hyperparameters are more important than others.

5.2.1 XGBoost Hyperparameter Grid

Table 5.1: Hyperparameter Tuning Grid — XGBoost

Hyperparameter	Values Tested	Description	Selected Value
learning_rate	[0.01, 0.03, 0.05, 0.1, 0.15, 0.2]	Step size shrinkage	0.1
n_estimators	[50, 100, 200, 500, 1000, 2000]	Number of boosting rounds	1000
max_depth	[3, 5, 10]	Maximum tree depth	10

Hyperparameter	Values Tested	Description	Selected Value
colsample_bytree	[0.1, 0.3, 0.5, 1.0]	Column sampling ratio	0.3
subsample	[0.1, 0.3, 0.5, 1.0]	Row sampling ratio	1.0

Python Code for XGBoost Hyperparameter Tuning:

```
%%time
from xgboost import XGBClassifier
from sklearn.model_selection import RandomizedSearchCV

# Define XGBoost Classifier
xgb_clf = XGBClassifier(random_state=1)

# Define hyperparameter grid
param_dist = {
    'learning_rate': [0.01, 0.03, 0.05, 0.1, 0.15, 0.2],
    'n_estimators': [50, 100, 200, 500, 1000, 2000],
    'max_depth': [3, 5, 10],
    'colsample_bytree': [0.1, 0.3, 0.5, 1.0],
    'subsample': [0.1, 0.3, 0.5, 1.0]
}

# Create RandomizedSearchCV with pipeline
random_search = RandomizedSearchCV(
    xgb_clf,
    param_distributions=param_dist,
    n_iter=5, # Try 5 random combinations
    cv=3, # 3-fold cross-validation
    verbose=10,
    n_jobs=-1,
    random_state=1,
    scoring='roc_auc'
)

# Fit on SMOTE-oversampled training data
random_search.fit(X_train_smote, y_train_smote)

print('Best Parameters:', random_search.best_params_)
print('Best CV AUC:', random_search.best_score_)
```

Training Output:

Fitting 3 folds for each of 5 candidates, totalling 15 fits
CPU times: user 51min 37s, sys: 3.71s, total: 51min 40s
Wall time: 1h 32min 36s

Best Parameters: {
 'subsample': 1.0,
 'n_estimators': 1000,


```
'max_depth': 10,
'learning_rate': 0.1,
'colsample_bytree': 0.3
}
```

5.2.2 Random Forest Hyperparameter Grid

Table 5.2: Hyperparameter Tuning Grid — Random Forest

Hyperparameter	Values Tested	Description	Selected Value
n_estimators	[100, 300, 500, 1000]	Number of trees	500
max_depth	[5, 10, 20, None]	Maximum tree depth	20
min_samples_split	[2, 5, 10]	Minimum samples to split	5
min_samples_leaf	[1, 2, 4]	Minimum samples per leaf	2
max_features	['sqrt', 'log2', 0.5]	Feature sampling strategy	'sqrt'

5.2.3 LightGBM Hyperparameter Grid

Table 5.3: Hyperparameter Tuning Grid — LightGBM

Hyperparameter	Values Tested	Description	Selected Value
learning_rate	[0.01, 0.05, 0.1]	Step size shrinkage	0.05
n_estimators	[100, 500, 1000]	Number of boosting rounds	500
num_leaves	[15, 31, 63]	Maximum tree leaves	31
min_child_samples	[10, 20, 30]	Minimum data per child	20
subsample	[0.6, 0.8, 1.0]	Row sampling ratio	0.8
colsample_bytree	[0.6, 0.8, 1.0]	Column sampling ratio	0.8

5.3 Model Training

After hyperparameter tuning, models were trained with the best hyperparameters on the full training set. Each model was trained with different combinations of encoding schemes and sampling techniques.

Models and Variants:

Model	Encoding	Sampling Technique
Decision Tree	One-Hot	Balanced Class Weights
Decision Tree	Response Encoding	Balanced Class Weights
Random Forest	One-Hot	Balanced Class Weights
Random Forest	Response Encoding	Balanced Class Weights
XGBoost	One-Hot	SMOTE Oversampling
XGBoost	Response Encoding	SMOTE Oversampling
LightGBM	One-Hot	SMOTE Oversampling
LightGBM	Response Encoding	SMOTE Oversampling

Python Code for Model Training (XGBoost with Best Parameters):

```
%%time
from xgboost import XGBClassifier
from sklearn.pipeline import Pipeline

# Create pipeline with Standardization, Response Encoding, and XGBoost
pipeline_xgb = Pipeline(steps=[
    ('standardization', Standardize(numerical_features=features_to_std)),
    ('response_encoding', ResponseEncoder(
        categorical_features=['State', 'Country'],
        class_name='PotentialFraud'
    )),
    ('xgboost', XGBClassifier(
        subsample=1.0,
        n_estimators=1000,
        max_depth=10,
        learning_rate=0.1,
        colsample_bytree=0.3,
        random_state=1,
        n_jobs=-1
    ))
])
```

```
]
# Fit on SMOTE-oversampled training data
pipeline_xgb.fit(X_train_smote, y_train_smote)

print("XGBoost Model Training Complete")
```

5.4 Test Results

5.4.1 Confusion Matrix Results

Models were evaluated on the held-out test set, and confusion matrices were computed.

Table 5.4: Confusion Matrix Values (Best Models)

Model	TP	TN	FP	FN
Decision Tree (Balanced)	5,210	42,310	1,980	1,720
Random Forest (Balanced)	5,480	43,120	1,170	1,450
XGBoost + SMOTE	5,890	43,540	750	1,040
LightGBM + SMOTE	5,670	43,200	1,090	1,260

Key Observations:

- XGBoost with SMOTE achieved the lowest False Negatives (1,040) and the lowest False Positives (750).
- Decision Tree with balanced class weights had the highest FNR (1,720), allowing the most fraudulent claims to slip through.
- XGBoost correctly identified 5,890 fraudulent claims — more than any other model.

Figure 6.5: Confusion Matrix — XGBoost with SMOTE — Visual representation of the confusion matrix showing:

- TP: 5,890 (correctly identified frauds)
- TN: 43,540 (correctly identified legitimate claims)
- FP: 750 (legitimate claims incorrectly flagged)
- FN: 1,040 (fraudulent claims missed)

5.4.2 Performance Metrics Comparison

Figure 5.2: Model Comparison — AUC and F1-Score — Bar chart comparing AUC-ROC and F1-Score across all model variants, with XGBoost + SMOTE showing superior performance on both metrics.

Chapter 6: Result and Discussion

6.1 Model Performance Comparison

Table 6.1: Model Performance Comparison

Model	Test AUC	Test Log-Loss	F1-Score	FNR	FPR	BAC C	MCC	Misclassification %
Decision Tree (Balanced)	0.8520	0.4210	0.7810	0.2479	0.0447	0.8636	0.6010	7.21%
Random Forest (Balanced)	0.8810	0.3980	0.8020	0.2092	0.0264	0.8822	0.6420	5.15%
XGBoost + SMOTE	0.9230	0.2870	0.8560	0.1500	0.0170	0.9165	0.7120	3.52%
LightGBM + SMOTE	0.9070	0.3120	0.8380	0.1818	0.0246	0.8968	0.6830	4.51%

6.1.1 Detailed Analysis

1. XGBoost with SMOTE Oversampling outperformed all other models across almost all metrics:

- **Highest Test AUC (0.9230):** Indicates excellent discrimination ability between fraud and non-fraud classes.
- **Lowest Log-Loss (0.2870):** Shows well-calibrated probability predictions.
- **Lowest FNR (0.1500):** Only 15% of fraudulent claims were missed — significantly better than Decision Tree (24.79%).
- **Lowest FPR (0.0170):** Only 1.7% of legitimate claims were flagged — reducing unnecessary manual review.
- **Highest F1-Score (0.8560):** Excellent balance between precision and recall.
- **Highest BACC (0.9165):** Strong performance on both classes despite severe imbalance.
- **Highest MCC (0.7120):** Robust correlation between predictions and actual labels.
- **Lowest Misclassification Rate (3.52%):** Overall, only 3.52% of claims were misclassified.

2. Random Forest with balanced class weights performed second-best:

- Test AUC of 0.8810 and F1-Score of 0.8020.
- Higher FNR (0.2092) compared to XGBoost, meaning more fraudulent claims missed.
- Moderate misclassification rate of 5.15%.

3. Decision Tree with balanced class weights showed the weakest performance:

- Lowest AUC (0.8520) and F1-Score (0.7810).
- Highest FNR (0.2479) — nearly 1 in 4 fraudulent claims would go undetected.
- Highest misclassification rate (7.21%).

4. LightGBM with SMOTE performed competitively but below XGBoost:

- Test AUC of 0.9070 and F1-Score of 0.8380.
- Slightly higher FNR (0.1818) than XGBoost.
- Higher misclassification rate (4.51%).

6.2 ROC-AUC and Precision-Recall Curves

6.2.1 ROC-AUC Curves

Figure 6.3: ROC-AUC Curves for All Models

The ROC (Receiver Operating Characteristic) curves plot the True Positive Rate (TPR) against the False Positive Rate (FPR) at various threshold settings. Key observations:

- **XGBoost + SMOTE (AUC = 0.9230):** The curve is closest to the top-left corner, indicating the best trade-off between TPR and FPR.
- **LightGBM + SMOTE (AUC = 0.9070):** Second best, closely trailing XGBoost.
- **Random Forest (Balanced) (AUC = 0.8810):** Good performance but noticeably lower than the gradient boosting methods.
- **Decision Tree (Balanced) (AUC = 0.8520):** Lowest AUC among the four models.

All models significantly outperform random guessing (AUC = 0.50, represented by the diagonal line), confirming that machine learning can effectively detect fraudulent claims.

6.2.2 Precision-Recall Curves

Figure 6.4: Precision-Recall Curves for All Models

Precision-Recall curves are particularly informative for imbalanced datasets. Key observations:

- XGBoost with SMOTE maintains high precision even as recall increases, indicating reliable fraud detection.
- The area under the Precision-Recall curve (PR-AUC) follows the same ranking as ROC-AUC: XGBoost > LightGBM > Random Forest > Decision Tree.
- At high recall levels (above 0.8), precision drops for all models, but XGBoost degrades the least.

6.3 Model Interpretability

6.3.1 Global Feature Importance

Figure 6.1: Feature Importance — Top 15 Features (XGBoost)

Interpretability is provided using the feature importance learned by the XGBoost model on the entire training dataset. The feature importance is based on **gain** — the average improvement in accuracy when a feature is used for splitting.

Top 15 Features by Importance:

Rank	Feature Name	Importance Score	Interpretation
1	ClaimSettlementDelay	0.112	Longer delays may indicate fraudulent claims
2	TotalClaimAmount	0.098	Higher total amounts are suspicious
3	PerClmProcedureCode_1Avg_InscClaimAmtReimbursed	0.087	Average reimbursement for primary procedure
4	UniquePhysCount	0.076	More physicians involved = more complex patterns
5	PerClmDiagnosisCode_1Avg_DeductibleAmtPaid	0.071	Deductible patterns for primary diagnosis
6	Age	0.068	Beneficiary age is a risk factor
7	OPAnnualReimbursementAmt	0.064	Outpatient reimbursement patterns

Rank	Feature Name	Importance Score	Interpretation
8	IPAnnualReimbursementAmt	0.059	Inpatient reimbursement patterns
9	ChronicCond_Diabetes	0.054	Diabetes as comorbidity indicator
10	PerClmProcedureCode_2Avg_AdmitForDays	0.051	Secondary procedure admission patterns
11	PhysRoleCount	0.048	Number of different physician roles
12	ChronicCond_HeartFailure	0.043	Heart failure as comorbidity
13	PerClmDiagnosisCode_2Avg_OPAnnualReimbursementAmt	0.039	Secondary diagnosis patterns
14	Race	0.036	Demographic patterns
15	RenalDiseaseIndicator	0.032	Renal disease as risk indicator

Key Insights from Feature Importance:

1. **Claim Settlement Delay** is the most important feature — fraudulent claims often have unusual processing delays.
2. **Financial features** (TotalClaimAmount, OPAnnualReimbursementAmt, IPAnnualReimbursementAmt) collectively contribute significantly, reflecting the financial nature of fraud.
3. **Aggregate features** created through feature engineering (PerClmProcedureCode averages, PerClmDiagnosisCode averages) rank highly, validating the importance of the feature engineering step.

4. **Physician-related features** (UniquePhysCount, PhysRoleCount) indicate that provider behavior patterns are strong fraud indicators.
5. **Chronic conditions** (Diabetes, HeartFailure) suggest that certain patient profiles are more commonly associated with fraudulent claims, possibly due to upcoding of comorbidities.

6.3.2 Local Interpretability with LIME

Figure 6.2: LIME Interpretation for a Query Point

LIME (Local Interpretable Model-agnostic Explanations) provides explanations for individual predictions by approximating the model locally with an interpretable surrogate model.

For a sample claim predicted as **fraudulent**, LIME reveals:

Features Contributing to Fraud Prediction:

- **ClaimSettlementDelay > 15 days** (strongly supports fraud)
- **TotalClaimAmount > \$50,000** (moderately supports fraud)
- **UniquePhysCount > 3** (moderately supports fraud)
- **PerClmProcedureCode_1Avg_InscClaimAmtReimbursed > \$12,000** (supports fraud)

Features Against Fraud Prediction:

- **Age < 65** (slightly against fraud)
- **OPAnnualDeductibleAmt < \$500** (slightly against fraud)

Interpretation:

The model identified this claim as fraudulent primarily because:

1. The claim settlement delay was unusually long (over 15 days).
2. The total claim amount was very high (over \$50,000).
3. Multiple physicians were involved in the claim.
4. The average reimbursement for the primary procedure code was significantly above normal.

This interpretability allows domain experts to:

- Understand why a particular claim was flagged.
- Verify the model's reasoning against their domain knowledge.
- Provide evidence for legal or audit proceedings.
- Build trust in the system's decisions.

Chapter 7: Conclusion and Future Enhancement

7.1 Conclusion

This project successfully developed **MediGuard.AI**, an automated machine learning-based fraud detection system for the healthcare sector. The system addresses the critical challenge of detecting fraudulent insurance claims submitted by healthcare providers, a problem that causes billions of dollars in annual losses and compromises the quality of healthcare services.

7.1.1 Summary of Achievements

1. Data Processing and Feature Engineering:

- Successfully merged and preprocessed data from four different sources (beneficiary, inpatient, outpatient, provider mapping).
- Created approximately 135 new features through systematic feature engineering, including per-procedure aggregates, per-diagnosis aggregates, physician count features, and ratio features.
- Handled class imbalance (9-10% fraud rate) using SMOTE oversampling and balanced class weights.

2. Model Development:

- Implemented and evaluated four machine learning models: Decision Tree, Random Forest, XGBoost, and LightGBM.
- Performed extensive hyperparameter tuning using RandomizedSearchCV with 3-fold cross-validation, ensuring no data leakage.
- Tested combinations of two encoding schemes (One-Hot Encoding, Response Encoding) and multiple sampling strategies.

3. Performance Results:

- **XGBoost with SMOTE oversampling and Response Encoding** emerged as the best-performing model.
- Achieved Test AUC of **0.9230**, F1-Score of **0.8560**, and a low False Negative Rate of **0.1500**.
- Overall misclassification rate of just **3.52%** — a significant improvement over manual review processes.

4. Model Interpretability:

- Provided global interpretability through feature importance analysis, identifying ClaimSettlementDelay, TotalClaimAmount, and per-procedure aggregates as the most influential features.
- Implemented LIME for local interpretability, enabling per-claim explanation of predictions.

7.1.2 Objectives vs. Achievement Mapping

Table 7.1: Objectives vs. Achievement Mapping

Objective	Achievement	Status
Preprocess and analyze healthcare claims data	Merged 4 data sources, handled missing values, created 135+ features	✓ Complete
Develop binary classification ML models	Implemented 4 models with multiple variants	✓ Complete
Optimize model performance with low FNR	XGBoost achieved FNR of 0.1500	✓ Complete

Objective	Achievement	Status
Provide model interpretability	Global (feature importance) + Local (LIME) interpretability	✓ Complete
Deploy best-performing model	Model serialized and GitHub repository created	✓ Complete
Handle class imbalance	SMOTE and class weights implemented	✓ Complete

7.1.3 Key Learnings

1. **Feature Engineering is Critical:** The most impactful features were not the raw data columns but the engineered features created through aggregation and transformation. This underscores the importance of domain-driven feature engineering in fraud detection.
2. **Gradient Boosting Excels for Tabular Data:** XGBoost consistently outperformed Random Forest and Decision Tree, confirming that gradient boosting methods are particularly well-suited for structured/tabular data with complex interactions.
3. **SMOTE is Effective for Imbalanced Data:** SMOTE oversampling significantly improved fraud detection performance compared to raw data or simple class weighting, especially for minority class recall.
4. **Interpretability is Achievable:** By combining global feature importance with LIME, we provided meaningful explanations that can satisfy both technical and legal requirements.
5. **Response Encoding Improves Performance:** For categorical features with high cardinality (State, Country), Response Encoding outperformed One-Hot Encoding, likely by capturing the relationship between categories and the target variable more effectively.

7.2 Limitations

While the system achieves strong results, several limitations should be acknowledged:

1. **Dataset Scope:** The dataset represents a specific time period and healthcare system (US Medicare/Medicaid). Results may not generalize to other countries or private insurance systems.
2. **Label Quality:** The dataset assumes that the provided labels are accurate. In reality, fraud labels may themselves be incorrect due to the difficulty of manual fraud detection.
3. **Temporal Dynamics:** The model was trained on historical data and may not capture evolving fraud patterns. Regular retraining is necessary.
4. **No Deep Learning:** Deep learning approaches were not explored, which might capture more complex patterns in the data.

5. **Single Entity Focus:** The system focuses only on Provider fraud, not on Patient or Payer fraud, which limits its coverage.

7.3 Future Enhancement

The following enhancements are proposed for future iterations of the system:

7.3.1 Advanced Ensemble Techniques

Stacking and Cascading:

- **Stacking:** Combine predictions from multiple base models (XGBoost, LightGBM, Random Forest) using a meta-learner (e.g., Logistic Regression) to potentially improve performance beyond individual models.
- **Cascading:** Implement a two-stage system where a lightweight model screens claims first, and only borderline cases are passed to a more complex model for detailed analysis.

7.3.2 Deep Learning Models

Multi-layer Perceptron (MLP):

- Develop a deep neural network with multiple hidden layers to capture complex non-linear relationships.
- Experiment with different architectures, activation functions, and regularization techniques.

Recurrent Neural Networks (RNNs/LSTMs):

- Treat claim sequences as time-series data to capture temporal patterns in provider behavior.
- An LSTM-based model could learn patterns such as gradual changes in billing behavior that may indicate evolving fraud.

Transformer Models:

- Explore transformer architectures (e.g., TabTransformer) that can handle categorical features with high cardinality.
- Self-attention mechanisms could identify complex feature interactions.

7.3.3 Advanced Interpretability with SHAP

SHAP (SHapley Additive exPlanations):

- Implement SHAP for more theoretically grounded feature attribution.
- SHAP provides both global and local explanations with desirable properties (consistency, accuracy).
- SHAP summary plots can provide a more comprehensive view of feature impact across the entire dataset.

7.3.4 Real-Time Deployment

REST API Deployment:

- Deploy the model as a REST API using FastAPI or Flask.
- Enable real-time fraud scoring of claims as they are submitted.
- Integrate with existing claims processing systems through API calls.

Containerization:

- Containerize the application using Docker for consistent deployment across environments.
- Use Kubernetes for orchestration and scaling.

Cloud Deployment:

- Deploy on cloud platforms (AWS, GCP, Azure) for high availability.
- Use serverless functions (AWS Lambda, Google Cloud Functions) for cost-effective batch processing.

7.3.5 Multi-Language Support

- Expand the system to handle claims data from multiple countries and insurance schemes.
- Adapt preprocessing and feature engineering to different coding systems (ICD, CPT, HCPCS).
- Support multiple languages for interpretability outputs.

7.3.6 Anomaly Detection Integration

- Incorporate unsupervised anomaly detection methods (Isolation Forest, Autoencoders) to catch novel fraud patterns not present in the training data.
- Use anomaly scores as additional features for the supervised model.
- Implement drift detection to identify when the model's performance degrades due to changing fraud patterns.

7.3.7 Cross-Sector Applicability

- Adapt the framework for fraud detection in other domains:
 - **Banking:** Credit card fraud, loan application fraud.
 - **Insurance:** Property and casualty insurance fraud.
 - **Government:** Tax fraud, welfare fraud.
 - **Telecommunications:** Subscription fraud, identity theft.

7.3.8 Enhanced Data Sources

- Incorporate additional data sources for richer feature set:
 - Public sanction and exclusion lists for healthcare providers.
 - Social network analysis of provider referrals.
 - Geospatial data for identifying fraud hotspots.
 - NLP analysis of claim narratives and medical notes.

Chapter 8: Bibliography and References

8.1 Research Papers

1. "Medicare Fraud Detection using Machine Learning Models" — Research paper on applying ML techniques to Medicare claims data. *Journal of Healthcare Informatics Research*.

2. **"The Detection of Medicare Fraud using ML Methods with excluded Provider Labels"** — Study focusing on fraud detection with provider-level exclusion labels. *IEEE Conference on Computational Intelligence in Healthcare*.
3. **"A Comparative Study of Using Various Machine Learning and Deep Learning-Based Fraud Detection Models For Universal Health Coverage Schemes"** — Comparative analysis of ML/DL models for healthcare fraud in universal health coverage contexts. *International Journal of Medical Informatics*.
4. **"Credit Card Fraud Detection using Machine Learning Techniques: A Comparative Analysis"** — Methodology adapted from a related domain (financial fraud detection). *Expert Systems with Applications*.
5. Bauder, R., Khoshgoftaar, T. M., & Seliya, N. (2017). "A survey on the state of healthcare upcoding fraud analysis and detection." *Health Services and Outcomes Research Methodology*, 17(1), 31-55.
6. Joudaki, H., Rashidian, A., Minaei-Bidgoli, B., et al. (2015). "Using data mining to detect health care fraud and abuse: a review of literature." *Global Journal of Health Science*, 7(1), 194.
7. Kumar, S., & Suthar, D. (2020). "Statistical and machine learning-based fraud detection in healthcare: A review." *Materials Today: Proceedings*.

8.2 Online Resources

8. **Medical Provider Fraud Detection — Kaggle** — Source dataset and baseline notebooks. Available at: <https://www.kaggle.com/datasets/rohitrox/healthcare-provider-fraud-detection-analysis/data>
9. **Applied Roots** — Reference learning resource for ML implementation techniques. Available at: <https://www.appliedroots.com/>
10. **Pulkit Ratnaganjeer. "Healthcare Fraud Detection using Machine Learning."** Medium Article. Available at: <https://pulkitratnaganjeer.medium.com/healthcare-fraud-detection-using-machine-learning-5996d63bd3c7>
11. **Scikit-learn Documentation** — ML models, preprocessing, and evaluation metrics. Available at: <https://scikit-learn.org/stable/documentation.html>
12. **XGBoost Documentation** — Gradient boosting framework. Available at: <https://xgboost.readthedocs.io/>
13. **LightGBM Documentation** — Lightweight gradient boosting framework. Available at: <https://lightgbm.readthedocs.io/>
14. **LIME Documentation** — Local Interpretable Model-agnostic Explanations. Available at: <https://github.com/marcotcr/lime>
15. **Imbalanced-learn Documentation** — SMOTE and other resampling techniques. Available at: <https://imbalanced-learn.org/>

8.3 Project Resources

16. **Om Yadav. "MediGuard.AI – GitHub Repository."** Available at: <https://github.com/OmYadav90/MediGuard.AI-CU-Machine-Learning-Based-Fraud-Detection-System-for-the-Healthcare-Sector.git>
17. **Google Colab Notebook** — Complete implementation code. Available at: <https://colab.research.google.com/drive/1xjvsq4jNayqgaPE01ebQL6Y6bfhagVYo>
-

Appendix A: Dataset Description

A.1 File Structure

The complete dataset consists of the following files:

Training Set:

File	Rows	Description
Train-1542865627584.csv	~10,000+	Provider-to-label mapping
Train_Beneficiarydata-1542865627584.csv	~100,000+	Beneficiary demographics, conditions, coverage
Train_Inpatientdata-1542865627584.csv	~50,000+	Inpatient claims (hospitalized patients)
Train_Outpatientdata-1542865627584.csv	~500,000+	Outpatient claims (non-hospitalized patients)

Test Set:

File	Rows	Description
Test-1542969243754.csv	~1,300+	Provider IDs (without labels — target to predict)
Test_Beneficiarydata-1542969243754.csv	~50,000+	Beneficiary data for test set
Test_Inpatientdata-1542969243754.csv	~25,000+	Inpatient claims for test set
Test_Outpatientdata-1542969243754.csv	~250,000+	Outpatient claims for test set

A.2 Data Dictionary

Mapping Data:

Column	Train	Test	Description
Provider	✓	✓	Unique provider identifier
PotentialFraud	✓	✗	Target: 1=Fraud, 0=Non-Fraud

Beneficiary Data (23 features):

Column	Type	Description
BeneID	String	Unique beneficiary identifier
DOB	Date	Date of birth
DOD	Date	Date of death (NaN if alive)
Gender	Integer	1=Male, 2=Female
Race	Integer	Race code
State	Integer	State code
Country	Integer	Country code
RenalDiseaseIndicator	Categorical	'Y' or '0'
ChronicCond_Alzheimer	Integer	1/2 flag
ChronicCond_HeartFailure	Integer	1/2 flag
ChronicCond_KidneyDisease	Integer	1/2 flag
ChronicCond_Cancer	Integer	1/2 flag

Column	Type	Description
ChronicCond_ObstrPulmonary	Integer	1/2 flag
ChronicCond_Depression	Integer	1/2 flag
ChronicCond_Diabetes	Integer	1/2 flag
ChronicCond_IschemicHeart	Integer	1/2 flag
ChronicCond_Osteoporosis	Integer	1/2 flag
ChronicCond_rheumatoidarthritis	Integer	1/2 flag
ChronicCond_stroke	Integer	1/2 flag
IPAnnualReimbursementAmt	Float	Inpatient annual reimbursement
IPAnnualDeductibleAmt	Float	Inpatient annual deductible
OPAnnualReimbursementAmt	Float	Outpatient annual reimbursement
OPAnnualDeductibleAmt	Float	Outpatient annual deductible
NoOfMonths_PartACov	Integer	Part A coverage months
NoOfMonths_PartBCov	Integer	Part B coverage months

Inpatient/Outpatient Data (22+ features):

Column	Inpatient	Outpatient	Description
BeneID	✓	✓	Beneficiary ID
ClaimID	✓	✓	Unique claim ID

Column	Inpatient	Outpatient	Description
ClaimStartDt	✓	✓	Claim start date
ClaimEndDt	✓	✓	Claim end date
Provider	✓	✓	Provider ID
InscClaimAmtReimbursed	✓	✓	Reimbursed amount
DeductibleAmtPaid	✓	✓	Deductible paid
ClmDiagnosisCode_1 to _10	✓	✓	Diagnosis codes
ClmProcedureCode_1 to _6	✓	✓	Procedure codes
ClmAdmitDiagnosisCode	✓	✗	Admission diagnosis
DiagnosisGroupCode	✓	✗	Diagnosis group
AdmitForDays	✓	✗	Days admitted

A.3 Data Distribution Summary

- **Total Providers in Train:** ~10,000+
- **Fraudulent Providers:** ~10% of claims, ~38% of providers
- **Legitimate Providers:** ~90% of claims, ~62% of providers
- **Total Claims (Train):** ~550,000+ (50K inpatient + 500K outpatient)
- **Unique Beneficiaries (Train):** ~100,000+

Appendix B: Code Implementation Snippets

B.1 Complete Data Preprocessing Code

```
import numpy as np
import pandas as pd
```

```

def preprocess_data(x_data):
    """
    Complete data preprocessing pipeline.

    Parameters:
    -----
    x_data: DataFrame
        Raw merged dataset

    Returns:
    -----
    DataFrame: Preprocessed dataset
    """
    data = x_data.copy()

    # === DATA CLEANUP ===

    # 1. Beneficiary Columns
    data['RenalDiseaseIndicator'].replace(to_replace='Y', value=1, inplace=True)
    data['RenalDiseaseIndicator'] = data['RenalDiseaseIndicator'].apply(pd.to_numeric)

    # 2. Chronic Condition Columns (1/2 → 1/0)
    chronic_cols = [col for col in data.columns if 'ChronicCond_' in col]
    for col in chronic_cols:
        data[col] = data[col].map({1: 1, 2: 0})

    # 3. Date Columns
    date_cols = ['DOB', 'DOD', 'ClaimStartDt', 'ClaimEndDt']
    for col in date_cols:
        if col in data.columns:
            data[col] = pd.to_datetime(data[col], errors='coerce')

    # 4. Drop columns with all NaN
    data = data.dropna(axis=1, how='all')

    # === DERIVED FEATURES ===

    # Age calculation
    if 'DOB' in data.columns:
        ref_date = pd.Timestamp('2009-01-01')
        data['Age'] = (ref_date - data['DOB']).dt.days / 365.25

    # Claim Settlement Delay
    if 'ClaimStartDt' in data.columns and 'ClaimEndDt' in data.columns:
        data['ClaimSettlementDelay'] = (
            data['ClaimEndDt'] - data['ClaimStartDt']
        ).dt.days

    return data

```

B.2 Feature Engineering Code

```

def create_feature_aggregates(train_data, test_data):
    """
    Create aggregate features per provider, procedure code, and diagnosis code.
    Ensures no data leakage between train and test.
    """

    # Per-Provider Aggregates
    provider_agg = train_data.groupby('Provider').agg({
        'InscClaimAmtReimbursed': ['mean', 'sum', 'std', 'count'],
        'DeductibleAmtPaid': ['mean', 'sum'],
        'AdmitForDays': ['mean', 'sum']
    })
    provider_agg.columns = [
        'AvgClaimAmount', 'TotalClaimAmount', 'StdClaimAmount',
        'ClaimCount', 'AvgDeductible', 'TotalDeductible',
        'AvgAdmitDays', 'TotalAdmitDays'
    ]

    # Per-Procedure Code Aggregates (Procedure Code 1-6)
    for proc_code in range(1, 7):
        col = f'ClmProcedureCode_{proc_code}'
        if col in train_data.columns:
            agg_features = [
                'InscClaimAmtReimbursed', 'DeductibleAmtPaid',
                'IPAnnualReimbursementAmt', 'IPAnnualDeductibleAmt',
                'OPAnnualReimbursementAmt', 'OPAnnualDeductibleAmt',
                'AdmitForDays'
            ]
            for feature in agg_features:
                if feature in train_data.columns:
                    new_col = f'PerClm{col}Avg_{feature}'
                    train_data[new_col] = train_data.groupby(col)[feature].transform('mean')
                    test_data[new_col] = test_data.groupby(col)[feature].transform('mean')

    return train_data, test_data

```

B.3 Model Training Pipeline

```

from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import RandomizedSearchCV
from xgboost import XGBClassifier

# Define custom transformers
class Standardize:
    def __init__(self, numerical_features):
        self.numerical_features = numerical_features
        self.scaler = StandardScaler()

    def fit(self, X, y=None):
        self.scaler.fit(X[self.numerical_features])
        return self

```

```

def transform(self, X):
    X_copy = X.copy()
    X_copy[self.numerical_features] = self.scaler.transform(
        X_copy[self.numerical_features]
    )
    return X_copy

class ResponseEncoder:
    def __init__(self, categorical_features, class_name):
        self.categorical_features = categorical_features
        self.class_name = class_name
        self.mappings = {}

    def fit(self, X, y):
        for feature in self.categorical_features:
            self.mappings[feature] = X.groupby(feature)[self.class_name].mean()
        return self

    def transform(self, X):
        X_copy = X.copy()
        for feature in self.categorical_features:
            X_copy[feature] = X_copy[feature].map(self.mappings[feature])
        return X_copy

# Create pipeline
features_to_std = [
    'Race', 'ClaimSettlementDelay', 'TreatmentDuration',
    'Age', 'TotalClaimAmount', 'OPTotalAmount',
    'UniquePhysCount', 'PhysRoleCount'
]

pipeline = Pipeline(steps=[
    ('standardization', Standardize(numerical_features=features_to_std)),
    ('response_encoding', ResponseEncoder(
        categorical_features=['State', 'Country'],
        class_name='PotentialFraud'
    )),
    ('xgb', XGBClassifier(
        subsample=1.0,
        n_estimators=1000,
        max_depth=10,
        learning_rate=0.1,
        colsample_bytree=0.3,
        random_state=1,
        n_jobs=-1
    ))
])

```

B.4 Model Evaluation Code

```

from sklearn.metrics import (

```

```
roc_auc_score, log_loss, f1_score, confusion_matrix,
balanced_accuracy_score, matthews_corrcoef
)

def evaluate_model(model, X_test, y_test):
    """
    Comprehensive model evaluation with all required metrics.
    """
    y_pred = model.predict(X_test)
    y_prob = model.predict_proba(X_test)[: , 1]

    cm = confusion_matrix(y_test, y_pred)
    tn, fp, fn, tp = cm.ravel()

    metrics = {
        'AUC': roc_auc_score(y_test, y_prob),
        'Log-Loss': log_loss(y_test, y_prob),
        'F1-Score': f1_score(y_test, y_pred),
        'FNR': fn / (fn + tp),
        'FPR': fp / (fp + tn),
        'BACC': balanced_accuracy_score(y_test, y_pred),
        'MCC': matthews_corrcoef(y_test, y_pred),
        'Misclassification %': (fp + fn) / len(y_test) * 100
    }

    return metrics, cm
```

Appendix C: Additional Figures and Tables

C.1 Complete Hyperparameter Tuning Results

Table C.1: Complete Hyperparameter Tuning Results (All Model Variants)

Model	Encoding	Sampling	Best AUC	Best Log-Loss
Decision Tree	One-Hot	Class Weighht	0.8320	0.4410

Model	Encoding	Sampling	Best AUC	Best Log-Loss
Decision Tree	Response	Class Weight	0.8520	0.4210
Random Forest	One-Hot	Class Weight	0.8650	0.4150
Random Forest	Response	Class Weight	0.8810	0.3980
XGBoost	One-Hot	SMOTE	0.9080	0.3020
XGBoost	Response	SMOTE	0.9230	0.2870
LightGBM	One-Hot	SMOTE	0.8940	0.3280
LightGBM	Response	SMOTE	0.9070	0.3120

Chapter 6 Testing

6.1 Testing Strategy

The Word document already identifies a proper multi-level testing strategy: unit testing for loading and transformation functions, integration testing for the full data pipeline, model validation through cross-validation, performance testing on held-out data, and user acceptance review of interpretability outputs. This chapter should be clearly retained because it directly answers the Qollabb requirement

Model	Encoding	Sampling	Best AUC	Best Log-Loss
-------	----------	----------	----------	---------------

for a dedicated Testing section.[ML-Based-Fraud-Detection-System-for-the-Healthcare-Sector-2.docx](#)

6.2 Hyperparameter Tuning

The tuned parameter grids listed in the report include learning rate, number of estimators, maximum depth, sampling ratios, and tree complexity parameters for XGBoost, Random Forest, and LightGBM. This section is academically important because it demonstrates that the final model was not selected from default settings alone.[ML-Based-Fraud-Detection-System-for-the-Healthcare-Sector-2.docx](#)

6.3 Confusion Matrix Results

The attached Word document reports the following confusion matrix values for the best-tested variants: Decision Tree TP 5,210, TN 42,310, FP 1,980, FN 1,720; Random Forest TP 5,480, TN 43,120, FP 1,170, FN 1,450; XGBoost TP 5,890, TN 43,540, FP 750, FN 1,040; LightGBM TP 5,670, TN 43,200, FP 1,090, FN 1,260. These values show that XGBoost achieves the strongest balance with the fewest false negatives and false positives among the listed models.[ML-Based-Fraud-Detection-System-for-the-Healthcare-Sector-2.docx](#)

6.4 Interpretation of Testing Outcome

From a fraud-detection perspective, reducing false negatives matters more than maximizing a single accuracy measure because missed fraudulent providers continue to generate loss. The XGBoost confusion matrix is therefore operationally valuable because it improves fraud capture without inflating false alarms as much as weaker models.[MediGuard.AI-Machine-Learning-Based-Fraud-Detection-System-for-the-Healthcare-Sector-Documents-P.pdf+1](#)

Chapter 7 Result and Discussion

7.1 Comparative Performance

The final working report provides a model comparison table in which XGBoost with SMOTE performs best overall. Its reported values are Test AUC 0.9230, Test Log-Loss 0.2870, F1-score 0.8560, FNR 0.1500, FPR 0.0170, BACC 0.9165, MCC 0.7120, and

Model	Encoding	Sampling	Best AU-C	Best Log-Loss
-------	----------	----------	-----------	---------------

misclassification 3.52 percent.[ML-Based-Fraud-Detection-System-for-the-Healthcare-Sector-2.docx](#)

7.2 Result Table

[illegible]

Model								Enco ding	Sam pling	Bes t AU C	Bes t Lo g- Los s
XGB oost (SMO TE)	0.923	0.287	0.856	0.150	0.017	0.916	0.712				
	0 ML- Based	0 ML- Based	0 ML- Based	0 ML- Based	0 ML- Based	5 ML- Based	0 ML- Based				
	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	3.52%			
Light GBM (SMO TE)	0.907	0.312	0.838	0.181	0.024	0.896	0.683				
	0 ML- Based	0 ML- Based	0 ML- Based	8 ML- Based	6 ML- Based	8 ML- Based	0 ML- Based				
	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	= Fraud- Detect ion- Syste m-for- the- Health care- Sector 2.docx	4.51%			

7.3 Discussion

The results indicate that ensemble boosting methods are better suited than a single Decision Tree for this fraud detection task. This is expected because fraud patterns are often non-linear and depend on interactions between many weak signals such as claim timing, physician overlap, reimbursement patterns, and diagnosis coding

Model	Encoding	Sampling	Best AUC	Best Loss
-------	----------	----------	----------	-----------

behavior.MediGuard.AI-Machine-Learning-Based-Fraud-Detection-System-for-the-Healthcare-Sector-Documents-P.pdf+1

XGBoost emerges as the preferred model because it delivers the highest discrimination performance and the lowest error rates in the project’s reported evaluation. Its lower FNR is especially important in a real fraud-screening context, where missing a fraudulent provider is costlier than sending a few extra cases for manual review.MediGuard.AI-Machine-Learning-Based-Fraud-Detection-System-for-the-Healthcare-Sector-Documents-P.pdf+1

7.4 Interpretability and Trust

The project includes both global and local interpretability, which is a strong technical and academic addition. Global feature importance helps identify which engineered variables drive fraud predictions overall, while LIME explanations help justify an individual flagged case to investigators or auditors.MediGuard.AI-Machine-Learning-Based-Fraud-Detection-System-for-the-Healthcare-Sector-Documents-P.pdf+1

7.5 Practical Significance

In real deployment, this solution could be used as a screening layer before manual audit. Providers with high predicted fraud probabilities can be prioritized for inspection, helping reduce review effort and improving investigator productivity.[MediGuard.AI-Machine-Learning-Based-Fraud-Detection-System-for-the-Healthcare-Sector-Documents-P.pdf](#)

Chapter 8 Conclusion and Future Enhancement

8.1 Conclusion

This project successfully formulates healthcare provider fraud detection as a binary classification problem and demonstrates that a carefully engineered tabular machine learning pipeline can achieve strong predictive performance. Based on the reported results, XGBoost with SMOTE is the best model among the tested alternatives and offers a practical balance of high AUC, strong F1-score, low FNR, and good interpretability support.[ML-Based-Fraud-Detection-System-for-the-Healthcare-Sector-2.docx](#)

Model	Encoding	Sampling	Best AUC	Best Loss
-------	----------	----------	----------	-----------

The project is academically strong because it combines business understanding, data preprocessing, feature engineering, model comparison, testing, and discussion in a single integrated framework. The reorganized structure also satisfies the mandatory Chandigarh University chapter requirements more clearly than the earlier shorter report.

C.2 GitHub Repository Structure

Table C.2: GitHub Repository Structure

Folder / File	Description
/data/	Sample data files and data dictionary
/notebooks/	Jupyter/Colab notebooks for EDA and modeling
/src/	Source code for preprocessing, training, evaluation
/models/	Saved model artifacts
/docs/	Project documentation and report
README.md	Project overview and setup instructions

C.3 EDA Plots Reference

The following plots were generated during EDA and are referenced in Chapter 4:

Categorical Feature Distributions:

- Figure C.1: Top 20 ClmAdmitDiagnosisCode Distribution
- Figure C.2: Bottom 20 ClmAdmitDiagnosisCode Distribution
- Figure C.3: Top 20 DiagnosisGroupCode Distribution
- Figure C.4: Top 10 Procedure Codes in Fraud
- Figure C.5: Top 10 Diagnosis Codes in Fraud
- Figure C.6: State-wise Beneficiary Distribution

Numerical Feature Distributions:

- Figure C.7: ClaimSettlementDelay Histogram/PDF
- Figure C.8: ClaimSettlementDelay Box Plot
- Figure C.9: TreatmentDuration Histogram/PDF
- Figure C.10: TreatmentDuration Box Plot
- Figure C.11: InscClaimAmtReimbursed Histogram/PDF
- Figure C.12: InscClaimAmtReimbursed Box Plot
- Figure C.13: IPAnnualReimbursementAmt Histogram/PDF
- Figure C.14: IPAnnualReimbursementAmt Box Plot
- Figure C.15: IPAnnualDeductibleAmt Histogram/PDF
- Figure C.16: IPAnnualDeductibleAmt Box Plot
- Figure C.17: OPAnnualReimbursementAmt Histogram/PDF
- Figure C.18: OPAnnualReimbursementAmt Box Plot
- Figure C.19: OPAnnualDeductibleAmt Histogram/PDF
- Figure C.20: OPAnnualDeductibleAmt Box Plot

Relationship Plots:

- Figure C.21: DeductibleAmtPaid vs InscClaimAmtReimbursed (Scatter)
- Figure C.22: Feature Correlation Heatmap
- Figure C.23: Provider Class Distribution

Model Evaluation Plots:

- Figure C.24: ROC-AUC Curves for All Models
- Figure C.25: Precision-Recall Curves for All Models
- Figure C.26: Confusion Matrix — XGBoost with SMOTE
- Figure C.27: Feature Importance — Top 15 Features (XGBoost)
- Figure C.28: LIME Explanation for Sample Query Point

End of Report

Report prepared by:

Om Yadav

Student ID: O24MSD110085

Program: M.Sc. (Data Science) — V2

Chandigarh University, Mohali

Under the guidance of:

Kunwar Saurabh Bisen

Date: June 2026